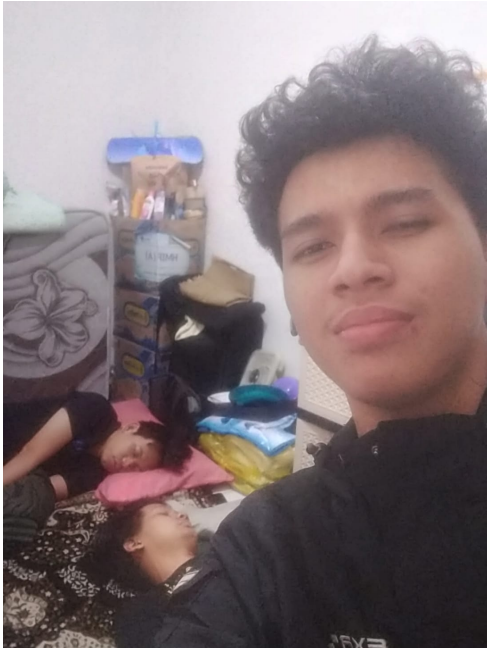
Laporan Tugas Besar 2

Pemanfaatan BFS dan DFS dalam Pencarian Recipe pada Permainan Little Alchemy 2

Disusun untuk memenuhi tugas mata kuliah IF2211 Strategi Algoritma pada Semester 2 (Genap) Tahun Akademik 2024/2025



Disusun oleh:

Refki Alfarizi	13523002
Razi Rachman Widyadhana	13523004
Muhammad Luqman Hakim	13523044

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Bab I: Deskripsi Masalah **1**

Bab II: Algoritma BFS dan DFS dalam Pencarian Recipe **2**
2.1. Penjejalahan Graf 2
2.2. Algoritma *Breath First Search* 2
2.3. Algoritma *Depth First Search* 3
2.4. Nani ga Suki? Website 4

Bab III: Abstraksi Permasalahan **5**
3.1. Element 5
3.2. Recipes 5
3.3. Pemetaan menjadi elemen-elemen BFS dan DFS 7
3.4. *Algoritma Penyelesaian Masalah* 8
3.5. Arsitektur Web 9
3.5.1 Gambaran Umum Arsitektur 9
3.5.2 *Frontend* 10
3.5.3 *Backend* 10
3.5.4 Interaksi *Frontend-Backend* 10
3.5.5 Dockerization 11
3.5.6 *Deployment* 11

Bab IV: Implementasi dan Pengujian **12**
4.1. Implementasi Program dengan `Next.js` dan `Golang` 12
4.1.1 Package `scraper` 12
4.1.2 Package `search` 17
4.2. Alur Program 30
4.3. Eksperimen 31
4.4. Analisis dan Pembahasan 37

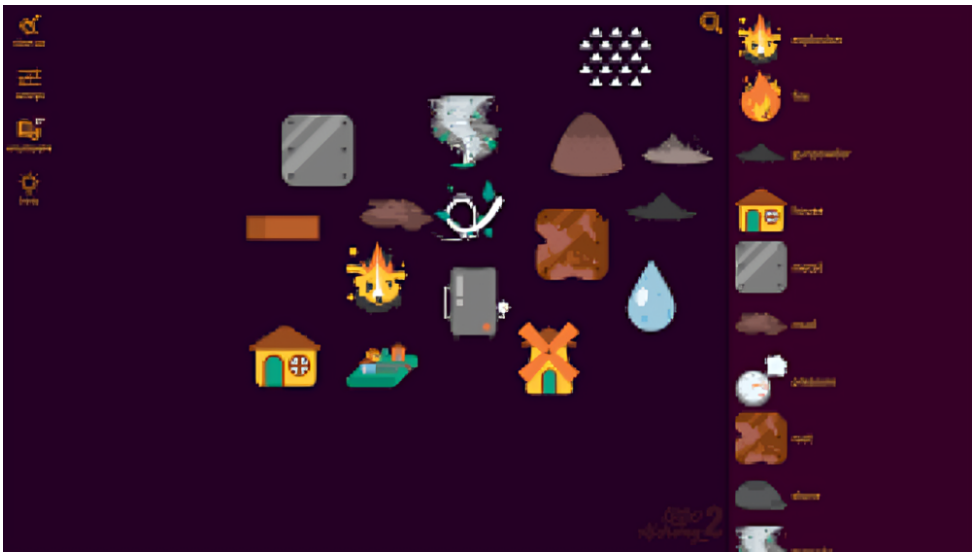
Bab V: Penutup **38**
5.1. Kesimpulan 38
5.2. Saran 38
5.3. Refleksi 39

Lampiran **40**

Referensi **41**

Bab I
Deskripsi Masalah

Little Alchemy 2 merupakan permainan berbasis *web*/aplikasi yang dikembangkan oleh Recloak yang dirilis pada tahun 2017, permainan ini bertujuan untuk membuat 720 elemen dari 4 elemen dasar yang tersedia yaitu *air*, *earth*, *fire*, dan *water*. Permainan ini merupakan sekuel dari permainan sebelumnya yakni Little Alchemy 1 yang dirilis tahun 2010.



Gambar 1: Permainan Little Alchemy 2 (Sumber: Little Alchemy Wiki)

Mekanisme dari permainan ini adalah pemain dapat menggabungkan kedua elemen dengan melakukan drag and drop, jika kombinasi kedua elemen valid, akan memunculkan elemen baru, jika kombinasi tidak valid maka tidak akan terjadi apa-apa. Permainan ini tersedia di web browser, Android atau iOS.

Komponen-komponen dari permainan ini antara lain:

1. **Elemen dasar:**

   Dalam permainan Little Alchemy 2, terdapat 4 elemen dasar yang tersedia yaitu *air*, *earth*, *fire*, dan *water*. Elemen-elemen dasar tersebut nanti akan di-combine menjadi elemen turunan yang berjumlah 720 elemen.

2. **Elemen turunan:**

   Terdapat 720 elemen turunan yang dibagi menjadi beberapa tier tergantung tingkat kesulitan dan banyak langkah yang harus dilakukan. Setiap elemen turunan memiliki recipe yang terdiri atas elemen lainnya atau elemen itu sendiri.

3. **Combine Mechanism:**

   Untuk mendapatkan elemen turunan pemain dapat melakukan combine antara 2 elemen untuk menghasilkan elemen baru. Elemen turunan yang telah didapatkan dapat digunakan kembali oleh pemain untuk membentuk elemen lainnya.

Bab II

Algoritma BFS dan DFS dalam Pencarian Recipe

2.1. Penjejalahan Graf

Algoritma traversal graf adalah algoritma yang mengunjungi simpul-simpul di dalam graf dengan cara yang sistematik untuk melakukan pencarian solusi persoalan yang direpresentrasikan dengan graf.

Algoritma Pencarian Solusi Berbasis Graf terbagi menjadi dua kelompok besar:

1. **Tanpa informasi (uninformed/blind search)**

   Tidak ada informasi tambahan yang disediakan. Contoh: BFS, DFS, *Depth Limited Search*, *Iterative Deepening Search*, *Uniform Cost Search*.

2. **Dengan informasi (informed Search)**

   Pencarian berbasis heuristik untuk mengetahui *non-goal state* yang "lebih menjanjikan" daripada yang lain. Contoh: Best First Search dan A*

Pada Tugas Besar 2 ini, akan digunakan BFS dan DFS sebagai algoritma penjelajahan graf dalam Pencarian *Recipe* pada permainan Little Alchemy 2.

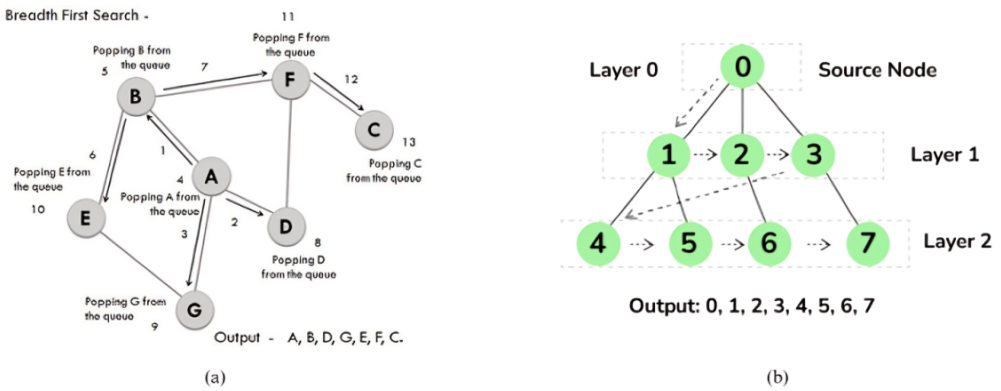
2.2. Algoritma *Breath First Search*

Algoritma *Breadth First Search* adalah algoritma pencarian yang melebar. Dimulai dari sebuah simpul dan menelusuri semua simpul yang berdekatan. Setelah semua simpul yang berdekatan dikunjungi, maka simpul-simpul yang berdekatan tersebut akan ditelusuri. Dengan interpretasi lain, menelusuri semua simpul pada kedalaman n terlebih dahulu sebelum menelusuri simpul pada kedalaman $n + 1$.

Kendala dari algoritma ini mungkin muncul saat melakukan pencarian pada struktur data *graph*. Tidak seperti *tree*, *graph* mungkin berisi siklus sehingga mungkin untuk sampai ke simpul yang sama lagi. Oleh karena itu, algoritma ini memanfaatkan struktur data *queue* dan *look-up table* dalam prosesnya untuk melacak simpul anak yang ditemukan tetapi belum ditelusuri.

Secara algoritmik, tahapan BFS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul sumber v yang diberikan ke dalam *queue* dan tandai sebagai telah dikunjungi.
2. **Eksplorasi:** Saat *queue* tidak kosong:
   - Keluarkan suatu simpul dari *queue* dan kunjungi simpul tersebut
   - Masukkan tetangga ke dalam *queue* untuk setiap tetangga yang belum dikunjungi dari simpul yang dikeluarkan.
   - Tandai simpul yang dikeluarkan sebagai telah dikunjungi.
3. **Basis:** Ulangi langkah 2 hingga *queue* kosong atau ditemukan simpul target.



Gambar 2: BFS pada (a) *Undirected Graph* dan (b) *Tree*

Biasanya, algoritma ini dapat digunakan untuk mencari jarak terpendek antara dua *node* dengan asumsi jarak antar *node* sama atau graf tidak berbobot. Di balik kelebihan dari solusi yang diberikan dijamin optimal/terpendek, terdapat kekurangan dari BFS yang membutuhkan memori yang cukup banyak karena menyimpan semua *simpul* yang ada.

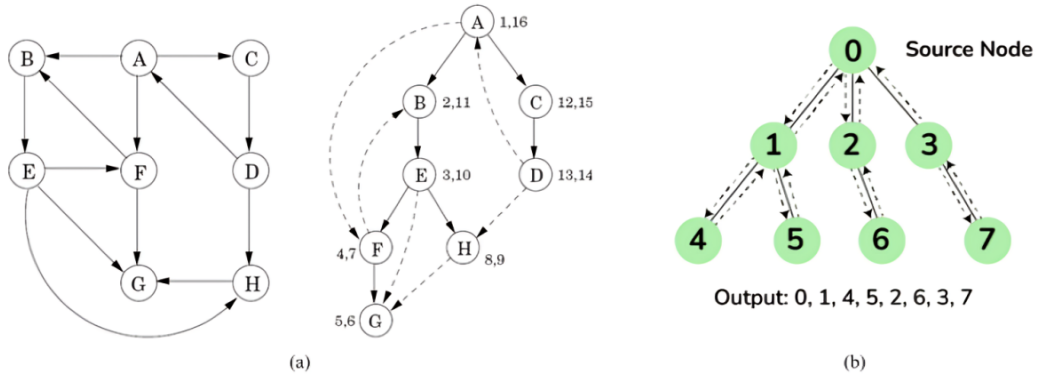
2.3. Algoritma *Depth First Search*

Algoritma *Depth First Search* adalah algoritma pencarian yang mendalam. Dimulai dari simpul awal dilanjutkan dengan hanya mengunjungi simpul anak paling kiri pada tingkat selanjutnya dan dilanjutkan pada node sebelah kanan. Jika solusi ditemukan maka tidak diperlukan proses *backtracking* yaitu penelusuran balik untuk mendapatkan jalur yang diinginkan.

Sama seperti BFS, algoritma ini terdapat kendala yang mungkin muncul saat melakukan pencarian pada struktur data *graph*. Oleh karena itu, algoritma ini memanfaatkan struktur data *stack* dan *look-up table* dalam prosesnya untuk melacak simpul anak yang ditemukan tetapi belum ditelusuri.

Secara algoritmik, tahapan DFS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul sumber v yang diberikan ke dalam *stack* dan tandai sebagai telah dikunjungi.
2. **Eksplorasi:** Saat *stack* tidak kosong:
   - Keluarkan suatu simpul dari *stack* dan kunjungi simpul tersebut.
   - Kunjungi simpul w yang bertetangga dengan simpul v dan masukkan ke *stack*
   - Ulangi sampai mencapai simpul u yang tidak memiliki tetangga yang belum dikunjungi.
3. **Backtrack:** Ketika mencapai simpul u, lakukan pencarian dirunut-balik (backtrack) ke simpul terakhir yang dikunjungi sebelumnya dan mempunyai simpul w yang belum dikunjungi.
4. **Basis:** Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi atau ditemukan simpul target.

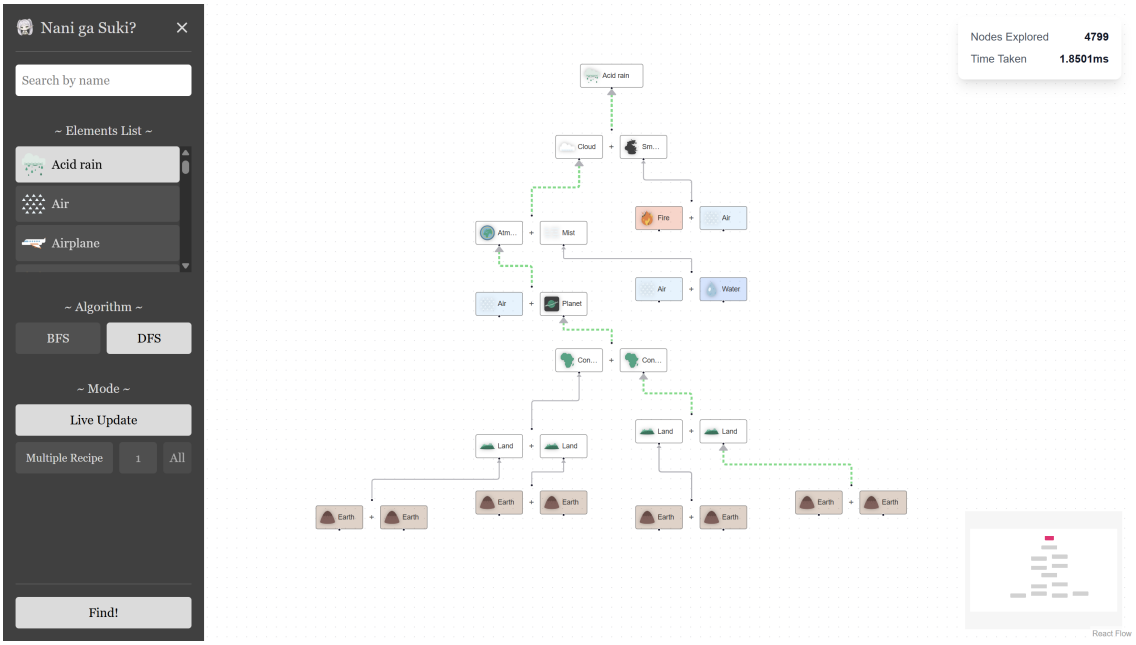


Gambar 3: DFS pada (a) *Directed Graph* dan (b) *Tree*

Dalam *directed acyclic graphs* (DAG), DFS dapat digunakan untuk melakukan pengurutan topologi, yang sangat penting dalam penjadwalan dan manajemen *prerequisites* (prasyarat). Di balik kelebihan dari penggunaan memori yang relatif lebih kecil, kekurangan dari DFS adalah tidak ada jaminan untuk menemukan solusi yang terbaik (tidak optimal).

2.4. Nani ga Suki? Website

Nani ga Suki? adalah website untuk mencari resep-resep dari elemen-elemen yang ada di permainan Little Alchemy 2 yang menggunakan pendekatan BFS dan DFS. Pengguna dapat memilih berbagai pilihan pencarian, antara lain pilihan pencarian resep menggunakan algoritma DFS/BFS dan pencarian satu/banyak resep. Pengguna juga memiliki pilihan untuk mendapatkan *live update* dari pencarian resep. Hasil resep ditampilkan sebagai *tree* yang interaktif.



Gambar 4: Cuplikan Laman Utama Nani ga Suki?

Bab III

Abstraksi Permasalahan

Seperti yang telah didefinisikan sebelumnya, algoritma BFS dan DFS akan melakukan pencarian pada struktur data *graph* ataupun *tree* sehingga fokus utama dari abstraksi permasalahan Tugas Besar 2 kali ini adalah tahapan cara mengonversi Recipe pada permainan Little Alchemy 2 menjadi suatu struktur data yang bisa dilakukan BFS dan DFS.

Dengan meninjau masing-masing komponen permainan terlebih dahulu, akan memudahkan proses abstraksi untuk penggunaan algoritma BFS dan DFS dalam Pencarian Recipe ini.

3.1. Element

Element dalam Little Alchemy 2 menggambarkan "bahan baku" yang merepresentasikan aspek-aspek alam, budaya, dan ilmu pengetahuan dalam wujud objek digital. Terdapat 4 *element* dasar seperti udara, air, tanah, dan api yang berfungsi sebagai unit dasar yang bersifat atomik di dalam ekosistem permainan.

Setiap *element* di Little Alchemy 2 dikelompokkan ke dalam tingkatan (*tier*) berdasarkan *tier element* minimum yang dibutuhkan untuk menciptakannya. Misalnya, *element* dengan *tier* 4 dapat dibentuk dengan cukup menggabungkan *element* dengan *tier* 3 atau di bawahnya.

Gambar 5: *Tier* dalam *Element*

Total *element* pada Little Alchemy 2 sebenarnya berjumlah 720 *element*. Akan tetapi, 3 *element* dari 720 *element* tersebut tidak digunakan pada Tugas Besar 2 ini. Hal ini disebabkan *element* Time merupakan *element* unik yang terbuka ketika membuka 100 *element*. Kemudian, *element* Ruins dan Archeologist jika tidak digunakan karena memerlukan *element* Time sebagai *element* pembentuknya (sesuai QnA).

3.2. Recipes

Dari penjelasan sebelumnya, dapat disimpulkan bahwa *element* di Little Alchemy 2 adalah entitas yang dapat dipasangkan dengan *element* lain untuk

membentuk "*Recipe*" yang menghasilkan *element* baru. Setiap kombinasi baru membuka peluang bagi kombinasi selanjutnya, menciptakan efek bola salju eksplorasi yang tak terbatas.

Sebenarnya, suatu *element* mungkin saja memiliki *recipe* dengan tier yang sama atau bahkan lebih tinggi dari *tier* dari *element* tersebut. Contoh sederhananya seperti Water (*tier* 0) yang seharusnya menjadi *element* dasar, masih bisa dibentuk dengan Heat (*tier 2* dengan Snow (*tier 7*) dan Ice 9.

Gambar 6: *Recipe* dari Water

Akan tetapi, untuk mengurangi pencarian pada Tugas Besar ini hanya digunakan *recipe* yang berisi *element-element* pembentuk dengan *tier* lebih rendah dari *elemen* yang ingin dibentuk (sesuai dengan revisi spesifikasi).

- Elemen yang digunakan pada suatu *recipe* harus berupa elemen dengan ***tier* lebih rendah** dari elemen yang ingin dibentuk.

Gambar 7: Revisi Spesifikasi H-4

Hasil dari *filtering* cukup mengagetkan karena terdapat *element* yang menjadi tidak memiliki *recipe*. Selain itu, terdapat beberapa *element* yang mengalami pengurangan jumlah *recipe* secara signifikan.

Element	***Tier***	N ***recipes* sebelum**	N ***recipes* sesudah**
Garage	11	40	4
Hangar	6	20	2
Airplane	9	38	1
Egg	8	35	1
Idea	8	31	1
Meat	8	41	1
Clock	7	6	0
Death	7	4	0
Dinosaur	9	1	0
Family Tree	9	2	0
Peat	10	2	0
Skeleton	9	2	0
Sloth	8	1	0
Tree	9	3	0

Perlu diperhatikan bahwa beberapa *element* ini merupakan *intermediate element*, maka memungkinkan *element* dengan *tier* tinggi menjadi tidak memiliki *recipe*. Maka, *insight* yang didapat dari *constraint* ini adalah perlu dilakukannya *pruning*/pemotongan ketika proses pencarian pada *Tree* berlangsung, bukan memfilternya pada saat *scraping*.

Kenapa tiba-tiba direvisi jadi resep apapun? Mungkin boleh kasih tahu alasannya? Terus apakah jawaban dari petanyaan 20 masih berlaku?	Agar DFS tidak terlalu berat. Iya masih berlaku.
Lanjuut 28, ini boleh ga kombinasi tiernya dipotong dari scraper aja?	Dipotong saat pembentukan Tree saja

Gambar 8: Jawaban QnA pertanyaan nomor 29

Selain itu, ditemukan juga beberapa *recipe* yang menggunakan *content pack supplements for the game's mobile releases* bernama Myths and Monsters. Hal. ini menyebabkan pencarian terputus karena *recipe* dari *element-element* dari *pack* tersebut berada di *webpage* yang terpisah.

Solusi dari asisten untuk permasalah ini adalah cukup dengan tidak memasukkan *recipe* tersebut saat melakukan proses *scraping*.

Untuk elemen yang merupakan dari MYTHS AND MONSTERS apakah dimasukkan ke data hasil scrapping? Jika ya, apakah pembuatan elemen MYTHS AND MONSTERS dimasukkan ke recipe karena recipe tidak dapat ditemukan pada https://little-alchemy.fandom.com/wiki/Elements_(Little_Alchemy_2) tapi harus masuk ke page nya sendiri https://little-alchemy.fandom.com/wiki/Zeus ? cth: zeus = deity + sky	Tidak

Gambar 9: Jawaban QnA pertanyaan nomor 10

Hal ini cukup memakan waktu karena harus melakukan *scraping* terlebih dahulu untuk *element-element* yang ada di *pack* tersebut, kemudian melakukan *filter* terhadap *element-element* tersebut. Terdapat beberapa *element* yang mengalami pengurangan jumlah recipe secara signifikan.

Element	***Tier***	N ***recipes*** **sebelum**	N ***recipes*** **sesudah**
Statue	10	34	17
Yeti	10	12	8
Angel	9	6	2

3.3. Pemetaan menjadi elemen-elemen BFS dan DFS

Setelah melakukan *cleaning* pada hasil *scraping* dari *recipes*, maka saatnya menentukan bagaimana bentuk dari struktur data yang akan digunakan dari permasalahan ini.

Bagaimana jika dimulai dengan ingin mencari suatu *full recipe* dari suatu *element* menuju *element* dasar? *Element* tersebut tentunya memiliki *recipes* yang berisi *element-element* pembentuknya terlebih dahulu.

Misalkan suatu simpul **target** v, akan memiliki simpul anak w sejumlah n banyak *recipes* yang dapat membentuknya. Akan tetapi, simpul anak tersebut tidak hanya mengandung satu *element*, melainkan *pair* dari 2 *element* pembentuk dalam satu *recipe*.

Maka, suatu simpul v akan berisi kedalaman simpul tersebut dan `pair<`element$_1$, element$_2$`>` yang mana masing-masing element$_1$ dan element$_2$ merupakan suatu simpul lagi berupa s_1 dan s_2 dengan isi simpul yang sama.

Untuk memudahkan pemahaman, berikut ilustrasinya.

Lantas bagaimana jika suatu *element* menjadi *ingredient* dari suatu *recipe* lagi nantinya? Untuk *handling* kasus ini, cukup sesimpel membentuk simpul baru walaupun dengan isi yang sama. Terus apa yang membedakannya dengan simpul sebelumnya? Dapat digunakan *identifier* ataupun dengan cek terlebih dahulu kedalamannya.

3.4. *Algoritma Penyelesaian Masalah*

Secara garis besar dari abstraksi yang telah dilakukan, digunakan struktur data *Tree* dengan simpul berisi kedalaman dan `pair<`element$_1$, element$_2$`>` yang element$_1$ dan element$_2$ sehingga sisi menyatakan bahwa simpul v tersebut dapat dibentuk dengan suatu *recipe* dengan simpul s_1 dan s_2 yang merupakan *ingredients* pada *recipe* tersebut.

Adapun tahapan algoritma BFS yang diimplementasikan adalah sebagai berikut.

1. Kelompokkan semua elemen ke dalam *bucket* berdasarkan nilai tier-nya.
2. Inisialisasi `memo` dengan elemen-elemen tier 0. Tambahkan setiap elemen dasar sebagai node tunggal dan tandai sebagai sudah ditelusuri.
3. Untuk setiap elemen pada tier tertentu, coba bentuk jalur baru dari elemen tersebut dan elemen-elemen dalam `memo`. Jika kombinasi valid dan jumlah jalur belum melebihi `maxPaths`, tambahkan jalur tersebut ke dalam memo.
4. Setelah semua tier selesai, periksa apakah elemen target tersedia dalam `memo`. Jika ya, ambil `maxPaths` jalur yang mengarah ke target tersebut.
5. Buat node target sebagai akar pohon.
6. Masukkan jalur-jalur elemen ke dalam pohon sebanyak jumlah yang diinginkan.

Adapun tahapan algoritma DFS yang diimplementasikan adalah sebagai berikut.

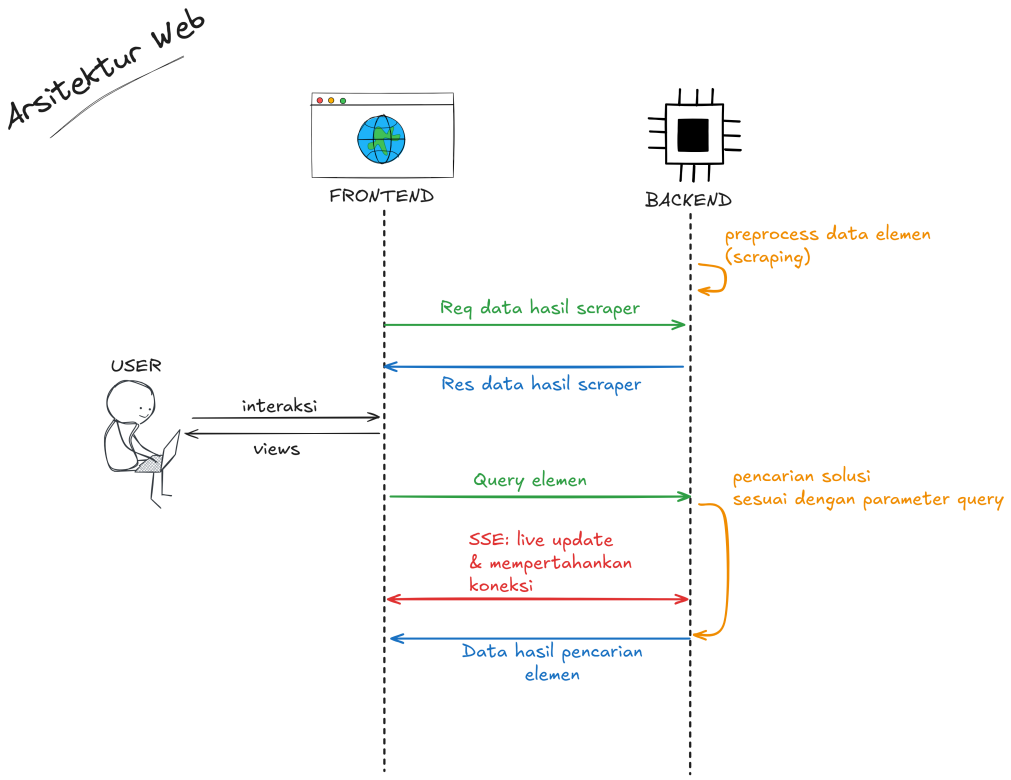
1. Verifikasi keberadaan elemen target dalam himpunan data. Jika elemen tidak ditemukan, proses dihentikan. Jika ditemukan, dibuat simpul akar sebagai representasi awal dari struktur solusi.
2. Kasus basis: Jika elemen target berada pada tier 0, pencarian resep sudah selesai, karena elemen tersebut adalah elemen dasar.
3. Kasus rekursi: Hitung jumlah jalur unik dari seluruh pasangan komponen yang memungkinkan. Kemudian, cari resep untuk kedua elemen dari pasangan-pasangan resep sebanyak jumlah resep yang diinginkan dengan DFS.
4. Gabungkan semua sub-pohon yang valid ke dalam simpul akar.

3.5. Arsitektur Web

Arsitektur sistem web merupakan kerangka yang mengatur hubungan antar-komponen dalam suatu aplikasi, mulai dari interaksi pengguna hingga pemrosesan data. Pada bab ini, akan dijelaskan secara komprehensif arsitektur *Nani ga Suki?* yang dibangun menggunakan Next.js + React pada sisi *frontend* dan Go pada sisi *backend*.

3.5.1 Gambaran Umum Arsitektur

Secara keseluruhan, sistem dibagi menjadi tiga komponen utama: browser pengguna, server *frontend*, dan server *backend*. Ketika pengguna membuka aplikasi, permintaan HTTP diterima oleh Next.js yang berjalan di server frontend. Berdasarkan jenis halaman, Next.js hanya akan memilih Client-Side Rendering (CSR). Jika halaman memerlukan data dinamis, Next.js akan meneruskan permintaan ke API backend yang dikembangkan dengan Go. Di sana, *request* diproses, akses basis data dilakukan, lalu hasilnya dikembalikan dalam format JSON.



Gambar 10: *Sequence Diagram* keseluruhan website

3.5.2 *Frontend*

Untuk sistem *frontend* tidak ada yang spesial. Penggunaan Next.js sebenarnya cukup *overkill* untuk proyek ini namun karena saran dari spesifikasi tugas maka tetap kami gunakan. Selain itu, karena website *Nani ga Suki?* hanya memiliki satu *route/page* maka fitur *App Routing* dan hal lainnya tidak terlalu berguna.

Untuk penggunaan React sendiri sempat menghambat pengembangan karena cara menggunakan *context/reference* yang cukup kompleks (terdapat *library* yang bisa digunakan untuk menyelesaikan masalah ini namun dirasa *overkill*).

3.5.3 *Backend*

Sistem *backend* menggunakan Go (sesuai spesifikasi tugas) dengan `net/http` sebagai *framework* untuk HTTP *Client and Server provider*. Pemilihan `net/http` (yang merupakan *Standard Library*) sebagai *framework* utama didasari oleh kebutuhan yang minim, yaitu *request* file dan Server-Sent Events (SSE) saja, sehingga penggunaan *framework* seperti `gin` atau `Echo` tidak diperlukan.

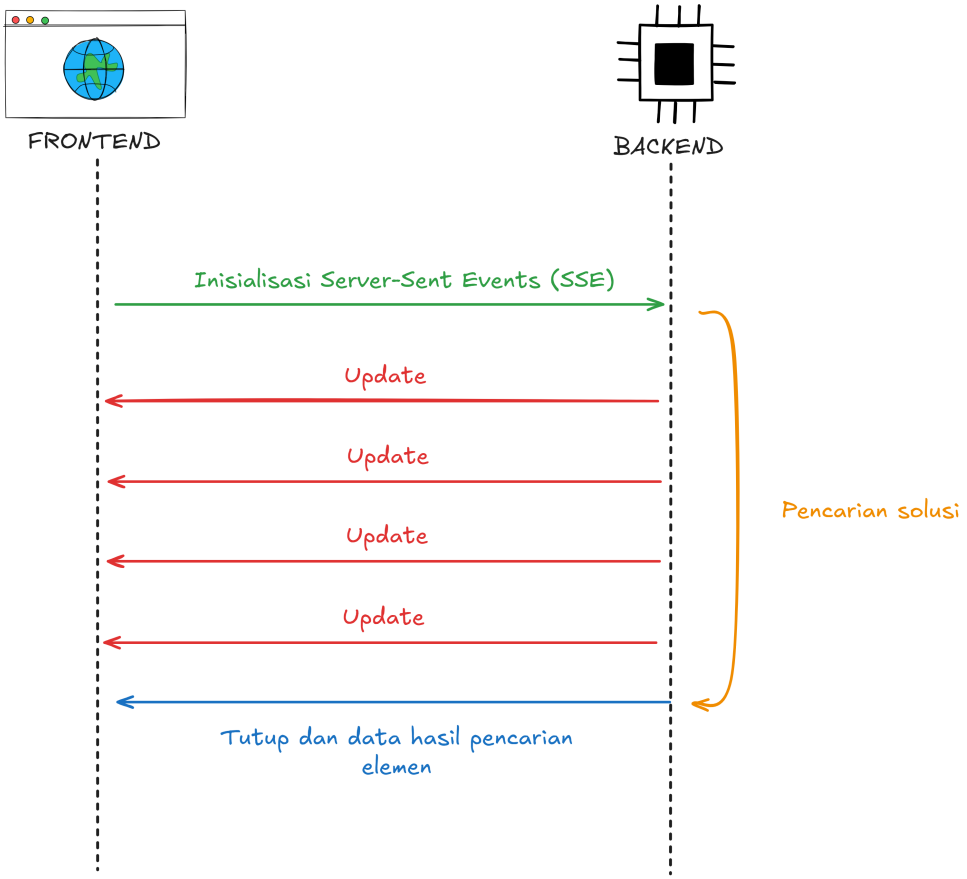
3.5.4 Interaksi *Frontend-Backend*

Ada dua interaksi yang dilakukan *frontend* dengan *backend*, yaitu *request* file hasil *scraping* dan *request* informasi *update* juga hasil akhir dari pencarian elemen.

Untuk *request* file hasil *scraping*, server cukup membaca file yang sudah tersimpan dan memberikan hasil bacaannya tersebut. *Scraping* dilakukan setiap server menyala, tetapi permintaan file hasil *scraping* tidak dilakukan setiap pengguna mengakses website. Hasil data dari *backend* disimpan di *local storage* pada browser sehingga mengurangi *bandwith* yang tidak diperlukan dan mendukung *green computing* sebagai informatikawan yang baik.

Lalu, untuk informasi *update*, *frontend* akan membuka komunikasi dengan protokol Server-Sent Events (SSE). Selama komunikasi terbuka, *backend* akan terus memberikan *update* dari proses pencarian elemen. Untuk mode pencarian *live update* maupun hasil langsung, SSE akan tetap digunakan tetapi perbedaannya untuk hasil langsung akan disertakan informasi keseluruhan *tree* sedangkan untuk *live update* tidak disertakan.

Rasionalisasi pemilihan SSE dibandingkan protokol lain seperti WebSocket adalah kembali lagi kepada keperluan projek ini yang hanya membutuhkan komunikasi satu arah dari server ke klien selama pencarian elemen.



Gambar 11: Diagram alur komunikasi dalam *frontend* dengan *backend* pencarian elemen

3.5.5 Dockerization

Untuk memudahkan *deployment* dan menjamin konsistensi lingkungan di seluruh tahapan (development, staging, production), aplikasi ini di-*containerize* menggunakan Docker. Setiap komponen *frontend* (Next.js/React) dan *backend* (Go) dibungkus dalam *image* Docker terpisah sehingga dependensi dan konfigurasi spesifik dapat diisolasi.

Untuk memudahkan proses *build*, digunakan `docker-compose` untuk mengorkestrasikan *image* Docker dari *frontend* dan *backend*.

3.5.6 *Deployment*

Untuk *deployment*, kami memilih untuk menggunakan Azure dari Microsoft sebagai penyewa Virtual Machine (VM) karena alasan kemudahan, keandalan, dan yang terutama kebaikannya atas *credit* gratis untuk kami para mahasiswa.

Virtual Machine yang disewa akan digunakan untuk menjalankan seluruh program (melalui *docker-compose* yang sudah dibahas sebelumnya) sehingga website yang kami kembangkan dapat berjalan setiap saat dan diakses oleh siapapun (untuk kemudahan akses, kami menyewa *domain* `http://nanigasuki.xyz`)

Bab IV
Implementasi dan Pengujian

4.1. Implementasi Program dengan `Next.js` dan `Golang`

Algoritma yang telah dijelaskan pada Bab 3 diimplementasikan dengan bahasa pemrograman `Golang` dan *framework* `Next.js`. Untuk program ini, sebenarnya terdapat beberapa *file*. Akan tetapi, tiga *package* utama yang berperan sebagai *core logic* program ini adalah `scraper`, `search`, dan .

Selain dari 3 *package* yang dibuat, program kami juga menggunakan *external library* untuk membantu implementasi, yaitu library `goquery` yang digunakan untuk membantu proses *scraping recipe* dari Website Wiki Little Alchemy 2. Kemudian ada juga library `reactflow` yang digunakan untuk memproses simpul-simpul yang telah terbentuk menjadi *Live Update* agar terlihat proses algoritma berjalan pada *frontend*.

4.1.1 Package `scraper`

```go
package scraper

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "os"
    "strconv"
    "strings"

    "github.com/PuerkitoBio/goquery"
)

/**
 *  ElementData holds recipes, image link, and tier for an element.
 */
type ElementData struct {
    Tier      int        `json:"tier"`
    ImageLink string     `json:"imageLink"`
    Recipes   [][2]string `json:"recipes"`
}

/**
 *  Run executes the scraper with optional Myths & Monsters update.
 *  If updateMAM is true, it rescans and caches the M&M dataset;
 *  otherwise loads from the existing cache.
 *  Returns the LA2 element map.
 */
func Run(updateMAM bool) map[string]ElementData {
    var mamMap map[string]ElementData
    const mamCache = "data/myths_and_monsters.json"
    mamURL := "https://little-alchemy.fandom.com/wiki/Elements_
```

```go
            (Myths_and_Monsters)"
    // 1) Myths & Monsters
    if updateMAM {
        mamMap = scrapePage(mamURL, "table.list-table")
        if err := writeJSON(mamCache, mamMap); err != nil {
            log.Fatalf("failed to write M&M cache: %v", err)
        }
        fmt.Printf("Cached %d Myths & Monsters elements\n",
        ↪  len(mamMap))
    } else {
        file, err := os.Open(mamCache)
        if err != nil {
            log.Fatalf("failed to open M&M cache: %v", err)
        }
        defer file.Close()
        decoder := json.NewDecoder(file)
        mamMap = make(map[string]ElementData)
        if err := decoder.Decode(&mamMap); err != nil {
            log.Fatalf("failed to load M&M cache: %v", err)
        }
        fmt.Printf("Loaded %d Myths & Monsters elements from
        ↪  cache\n", len(mamMap))
    }

    // Build filter of M&M names
    filter := make(map[string]struct{}, len(mamMap))
    for name := range mamMap {filter[name] = struct{}{}}

    // 2) Little Alchemy 2 raw scrape
    la2URL :=
    "https://little-alchemy.fandom.com/wiki/Elements_
    (Little_Alchemy_2)"
    laMap := scrapePage(la2URL,
    ↪  "table.list-table.col-list.icon-hover")

    // Filter out recipes containing any M&M element
    for name, data := range laMap {
        var cleaned [][2]string
        for _, pair := range data.Recipes {
            if _, skip := filter[pair[0]]; skip {continue
            }
            if _, skip := filter[pair[1]]; skip {
                continue
            }
            cleaned = append(cleaned, pair)
        }
        data.Recipes = cleaned
        laMap[name] = data
    }

    // Save raw LA2 output
    if err := writeJSON("data/recipes.json", laMap); err != nil {
        log.Fatalf("failed to write LA2 JSON: %v", err)
    }
    fmt.Printf("Wrote %d Little Alchemy 2 elements\n", len(laMap))

    return laMap
```

```go
}

/**
 *  writeJSON writes the provided data map to the given filepath in
 ↪  pretty JSON.
 */
func writeJSON(path string, data map[string]ElementData) error {
    file, err := os.Create(path)
    if err != nil {
        return err
    }
    defer file.Close()
    enc := json.NewEncoder(file)
    enc.SetIndent("", "  ")
    return enc.Encode(data)
}

/**
 *  scrapePage fetches the HTML from the given URL and extracts
 ↪  elements
 */
func scrapePage(url, tableSel string) map[string]ElementData {
    resp, err := http.Get(url)
    if err != nil {
        log.Fatalf("request error %s: %v", url, err)
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        log.Fatalf("bad status %d for %s", resp.StatusCode, url)
    }
    doc, err := goquery.NewDocumentFromReader(resp.Body)
    if err != nil {
        log.Fatalf("parse error %s: %v", url, err)
    }

    result := make(map[string]ElementData)
    doc.Find("span.mw-headline").Each(func(_ int, sel
    ↪  *goquery.Selection) {
        id, _ := sel.Attr("id")
        var tier int
        switch {
        case id == "Starting_elements":
            tier = 0
        case strings.HasPrefix(id, "Tier_"):
            parts := strings.Split(id, "_")
            if n, err := strconv.Atoi(parts[1]); err == nil {
                tier = n
            } else {
                return
            }
        default:
            return
        }

        tbl := sel.Closest("h3").NextAll().Filter(tableSel).First()
        if tbl.Length() == 0 {
            return
```

```go
        }

        tbl.Find("tbody tr").Each(func(i int, row
        ↪  *goquery.Selection) {
            if i == 0 {
                return
            }
            cell := row.Find("td").Eq(0)
            name :=
            ↪  strings.TrimSpace(cell.Find("a[title]").First().Text()
            if name == "" || name == "Time" || name == "Ruins" ||
            ↪  name == "Archeologist" {
                return
            }
            img, _ :=
            ↪  cell.Find("a.mw-file-description.image").Attr("href")

            var recipes [][2]string
            row.Find("td").Eq(1).Find("ul li").Each(func(_ int, li
            ↪  *goquery.Selection) {
                var ings []string
                li.Find("a[title]").Each(func(_ int, s
                ↪  *goquery.Selection) {
                    txt := strings.TrimSpace(s.Text())
                    if txt != "" && txt != "Time" && txt != "Ruins"
                    ↪  && txt != "Archeologist" {
                        ings = append(ings, txt)
                    }
                })
                if len(ings) >= 2 {
                    recipes = append(recipes, [2]string{ings[0],
                    ↪  ings[1]})
                }
            })

            result[name] = ElementData{Tier: tier, ImageLink: img,
            ↪  Recipes: recipes}
        })
    })

    return result
}
```

Web scraper digunakan untuk mengambil serta mengelola data elemen dari permainan *Little Alchemy 2*, termasuk ekspansi *Myths & Monsters*. Data yang diambil meliputi nama elemen, tautan gambar, *tier*, serta kombinasi resep pembentuk elemen. Informasi ini disimpan dalam format JSON untuk kebutuhan pemrosesan data lebih lanjut maupun penggunaan secara offline.

Struktur data utama dalam scraper ini adalah sebuah `struct` bernama `ElementData` yang memiliki tiga atribut, yaitu `Tier` yang merepresentasikan level kompleksitas elemen dalam permainan, `ImageLink` yang berisi URL gambar dari elemen, serta `Recipes`, yang merupakan

daftar pasangan kombinasi dua elemen yang digunakan untuk membentuk elemen tersebut. `Recipes` disimpan dalam bentuk `slice` dari array berukuran dua bertipe `string`.

Fungsi `Run` merupakan titik awal eksekusi utama dari program scraper ini. Fungsinya adalah untuk menghasilkan sebuah `map` yang memuat semua nama elemen dari permainan *Little Alchemy 2* beserta atribut-atributnya, seperti tier, tautan gambar, dan resep pembentuk. Fungsi ini bekerja dengan terlebih dahulu memuat data elemen dari ekspansi *Myths & Monsters* dan membentuk sebuah *filter* berisi nama-nama elemen dari ekspansi tersebut. Filter ini kemudian digunakan untuk menyaring hasil scraping data utama agar tidak mengandung resep yang melibatkan elemen ekspansi. Setelah itu, data elemen utama dari *Little Alchemy 2* diambil melalui proses scraping, lalu setiap resep dalam elemen-elemen tersebut diperiksa dan dibersihkan dari kombinasi yang mengandung elemen MAM. Hasil akhir dari proses ini adalah peta data yang telah difilter, yang kemudian disimpan ke dalam file `recipes.json` menggunakan fungsi `writeJSON` dan dikembalikan sebagai hasil dari fungsi `Run`.

Fungsi `scrapePage` bertanggung jawab untuk melakukan proses *web scraping* terhadap halaman wiki *Little Alchemy 2* berdasarkan URL dan selektor tabel yang diberikan sebagai parameter. Fungsi ini dimulai dengan melakukan permintaan HTTP (`GET`) ke URL target, kemudian memproses konten HTML yang diperoleh menggunakan pustaka `goquery`. Selanjutnya, fungsi menelusuri setiap bagian halaman yang mengandung elemen-elemen permainan berdasarkan tag `<span>` dengan kelas `mw-headline` yang biasanya memiliki atribut ID seperti `Starting_elements` atau `Tier_N`, yang menandakan tingkat atau kategori elemen tersebut. Dari setiap bagian ini, fungsi mencari tabel yang relevan untuk diolah.

Di dalam setiap tabel, fungsi mengambil nama elemen dari kolom pertama dan memproses daftar resep yang ditemukan pada kolom kedua. Setiap resep biasanya terdiri atas dua elemen penyusun, dan hanya resep yang valid serta tidak mengandung elemen-elemen tertentu yang dikecualikan secara eksplisit seperti `Time`, `Ruins`, dan `Archeologist` yang akan disimpan. Selain itu, fungsi juga mengambil tautan gambar elemen jika tersedia. Semua informasi ini kemudian dimasukkan ke dalam sebuah struktur data bertipe `ElementData`, yang selanjutnya disimpan dalam sebuah `map` dengan nama elemen sebagai kunci. Hasil akhir dari fungsi ini adalah sebuah `map[string]ElementData` yang merepresentasikan seluruh elemen yang ditemukan pada halaman tersebut beserta data lengkapnya.

4.1.2 Package `search`

File: `adt.go`

```go
package search

import (
        "fmt"
)

/**
 *  Target element we're searching for.
 */
type Target struct {
    Name        string   `json:"name"`
    Tier        int      `json:"tier"`
    Recipes     []Recipe `json:"recipes"`
    UniquePaths int      `json:"uniquePaths"`
    ID          uint64   `json:"id"`
}

/**
 *  Target is the final JSON tree.
 *  It contains the full recipe tree for a given element.
 */
type Tree = *Target

/**
 *  Element is a node in the final JSON tree.
 */
type Element struct {
    Name    string   `json:"name"`
    Tier    int      `json:"tier"`
    Recipes []Recipe `json:"recipes,omitempty"`
    ID      uint64   `json:"id"`
}


/**
 *  Recipe pairs two child Elements.
 */
type Recipe struct {
    Left  *Element `json:"left"`
    Right *Element `json:"right"`
}

/**
 *  Update events get streamed as we build.
 */
type Update struct {
    Stage       string // e.g. "startDFS", "startRecipe",
    ↪  "startBuildLeft", ...
    ElementName string
    Tier        int    // actual tier of ElementName
    RecipeIndex int    // which recipe of the *parent* we're on
    Info        string // any extra detail
```

```go
    ParentID uint64
    LeftID   uint64
    RightID  uint64

    LeftLabel  string
    RightLabel string
}

// Counts nodes in a recipe tree
func CountTreeNodes(el *Element) int {
    if el == nil {
        return 0
    }

    count := 1 // Count this node
    for _, r := range el.Recipes {
        count += CountTreeNodes(r.Left)
        count += CountTreeNodes(r.Right)
    }
    return count
}
```

Struktur data `Target` merepresentasikan elemen akhir yang dicari dalam proses pencarian jalur resep. Struktur ini memuat nama elemen, tingkat (*tier*) kemunculan dalam hierarki, daftar resep penyusunnya (`Recipes`), jumlah jalur unik yang ditemukan, serta sebuah identitas unik (`ID`). `Target` juga digunakan sebagai alias untuk tipe `Tree`, yang merepresentasikan pohon resep komposit secara keseluruhan.

Struktur `Element` merepresentasikan simpul dalam pohon hasil akhir. Setiap simpul menyimpan nama elemen, *tier*, dan daftar resep opsional jika elemen tersebut merupakan hasil komposit. Sama seperti `Target`, setiap `Element` memiliki `ID` unik untuk keperluan identifikasi selama proses konstruksi pohon.

Struktur `Recipe` menyatakan pasangan dua elemen penyusun, yaitu `Left` dan `Right`, yang digunakan untuk membentuk sebuah elemen baru. Objek dari tipe ini digunakan dalam koleksi resep milik `Element` atau `Target`, serta dalam proses pencarian dan pembangunan pohon komposisi.

Struktur `Update` mengirimkan status progres secara *real-time* selama proses konstruksi pohon, misalnya pada saat dimulainya traversal DFS atau saat membangun salah satu cabang resep. Struktur ini memuat informasi tahap pembangunan (`Stage`), nama dan tier elemen terkait, indeks resep saat ini, serta label dan ID dari simpul induk dan anaknya.

Fungsi `CountTreeNodes` digunakan untuk menghitung total jumlah simpul dalam suatu pohon `Element`. Fungsi ini bekerja secara rekursif dengan menjumlahkan 1 untuk setiap simpul yang dikunjungi, kemudian memanggil dirinya sendiri untuk simpul kiri dan kanan dari setiap resep yang ada. Fungsi ini mengembalikan jumlah keseluruhan node yang membentuk suatu pohon resep.

File: **bfs.go**

```go
/**
 *  BFS runs a bottom-up DP approach to find paths for the target
 ↪  element.
 *  It returns both the paths found and the number of nodes
 ↪  explored.
 */
func BFS(
    recipeMap map[string]scraper.ElementData,
    targetName string,
    maxPaths int,
) ([]*Element, uint64, error) {

    // Use a local node counter instead of a global variable
    var nodeCounter uint64 = 0

    byTier := make(map[int][]string, len(recipeMap))
    var tiers []int
    for name, data := range recipeMap {
        t := data.Tier
        if _, ok := byTier[t]; !ok {
            tiers = append(tiers, t)
        }
        byTier[t] = append(byTier[t], name)
    }
    sort.Ints(tiers)

    memo := make(map[string][]*Element, len(recipeMap))

    // Initialize tier 0 elements
    for _, name := range byTier[0] {
        memo[name] = []*Element{{Name: name, Tier: 0}}
        atomic.AddUint64(&nodeCounter, 1)
    }

    // Process higher tiers
    for _, tier := range tiers {
        if tier == 0 {
            continue
        }
        nextMemo := make(map[string][]*Element, len(byTier[tier]))
        var wg sync.WaitGroup
        var mu sync.Mutex
        sem := make(chan struct{}, runtime.NumCPU())

        for _, name := range byTier[tier] {
            wg.Add(1)
            sem <- struct{}{}
            go func(el string) {
                defer wg.Done()
                defer func() { <-sem }()

                atomic.AddUint64(&nodeCounter, 1)
                data := recipeMap[el]
                var examples []*Element
```

```go
                for _, pair := range data.Recipes {
                    leftList, lok := memo[pair[0]]
                    rightList, rok := memo[pair[1]]
                    if !lok || !rok || len(leftList) == 0 ||
                    ↪  len(rightList) == 0 {
                        continue
                    }
                    for _, L := range leftList {
                        for _, R := range rightList {
                            if len(examples) >= maxPaths {
                                break
                            }
                            if data.Tier <= L.Tier || data.Tier <=
                            ↪  R.Tier {
                                continue
                            }
                            atomic.AddUint64(&nodeCounter, 1)
                            examples = append(examples, &Element{
                                Name:    el,
                                Tier:    data.Tier,
                                Recipes: []Recipe{{Left: L, Right:
                                ↪  R}},
                            })
                        }
                        if len(examples) >= maxPaths {
                            break
                        }
                    }
                    if len(examples) >= maxPaths {
                        break
                    }
                }

                mu.Lock()
                nextMemo[el] = examples
                mu.Unlock()
            }(name)
        }

        wg.Wait()

        // merge this tier into memo
        for name, ex := range nextMemo {
            memo[name] = ex
        }
    }

    // Return up to maxPaths for target
    result, ok := memo[targetName]
    if !ok || len(result) == 0 {
        return nil, nodeCounter, nil
    }
    if len(result) > maxPaths {
        result = result[:maxPaths]
    }
    return result, nodeCounter, nil
}
```

```go
/**
 *  CreateFullTree takes the raw []*Element paths for the same
 ↪  target
 *  and builds a single *Target containing all their top-level
 ↪  recipes.
 *  It emits Update events when 'updates' is non-nil.
 */
func CreateFullTree(
    paths []*Element,
    updates chan<- Update,
    name string,
    nextID func() uint64,
) *Target {
    tgt := &Target{
        Name:        name,
        Tier:        0,
        Recipes:     nil,
        UniquePaths: len(paths),
        ID:          0,
    }
    if len(paths) > 0 {
        tgt.Name = paths[0].Name
        tgt.Tier = paths[0].Tier
    }

    // deep-clone function
    var clone func(src *Element) *Element
    clone = func(src *Element) *Element {
        node := &Element{
            Name:    src.Name,
            Tier:    src.Tier,
            Recipes: nil,
            ID:      0,
        }
        for _, r := range src.Recipes {
            L := clone(r.Left)
            R := clone(r.Right)
            node.Recipes = append(node.Recipes, Recipe{Left: L,
            ↪  Right: R})
        }
        return node
    }

    // helper: remove duplicated recipes by signature, merge their
    ↪  subtrees
    dedupeRecipes := func(recs []Recipe) []Recipe {
        uniq := make([]Recipe, 0, len(recs))
        seen := make(map[string]int, len(recs))
        for _, r := range recs {
            sig := r.Left.Name + "|" + r.Right.Name
            if idx, ok := seen[sig]; !ok {
                seen[sig] = len(uniq)
                uniq = append(uniq, r)
            } else {
                ex := &uniq[idx]
```

```go
                ex.Left.Recipes  = append(ex.Left.Recipes,
                ↪  r.Left.Recipes...)
                ex.Right.Recipes = append(ex.Right.Recipes,
                ↪  r.Right.Recipes...)
            }
        }
        return uniq
    }

    // -- merge top-level paths into tgt.Recipes (as before) --
    seen := make(map[string]int, len(paths))
    for _, p := range paths {
        root := clone(p)
        for _, rec := range root.Recipes {
            sig := rec.Left.Name + "|" + rec.Right.Name
            if idx, ok := seen[sig]; !ok {
                seen[sig] = len(tgt.Recipes)
                tgt.Recipes = append(tgt.Recipes, rec)
            } else {
                ex := &tgt.Recipes[idx]
                // merge both sides
                ex.Left.Recipes  = append(ex.Left.Recipes,
                ↪  rec.Left.Recipes...)
                ex.Right.Recipes = append(ex.Right.Recipes,
                ↪  rec.Right.Recipes...)
            }
        }
    }

    // assign root ID + initial update
    tgt.ID = nextID()
    if updates != nil {
        updates <- Update{
            Stage:       "startBFS",
            ElementName: tgt.Name,
            Tier:        tgt.Tier,
            Info:        fmt.Sprintf("root id=%d, merging %d
            ↪  paths", tgt.ID, len(paths)),
        }
    }

    type item struct {
        ParentID    uint64
        RecipeIndex int
        Node        *Element
    }
    queue := make([]item, 0, len(tgt.Recipes)*2)

    // seed queue with root's immediate children
    for i, rec := range tgt.Recipes {

        rec.Left.Recipes  = dedupeRecipes(rec.Left.Recipes)
        rec.Right.Recipes = dedupeRecipes(rec.Right.Recipes)

        rec.Left.ID  = nextID()
        rec.Right.ID = nextID()
        if updates != nil {
```

```
            updates <- Update{
                Stage:       "startRecipe",
                ElementName: tgt.Name,
                Tier:        tgt.Tier,
                ParentID:    tgt.ID,
                RecipeIndex: i,
                LeftID:      rec.Left.ID,
                RightID:     rec.Right.ID,
                LeftLabel:   rec.Left.Name,
                RightLabel:  rec.Right.Name,
                Info:        fmt.Sprintf("recipe %d under parent
                ↪  %d", i, tgt.ID),
            }
        }
        queue = append(queue,
            item{ParentID: rec.Left.ID, RecipeIndex: i, Node:
            ↪  rec.Left},
            item{ParentID: rec.Right.ID, RecipeIndex: i, Node:
            ↪  rec.Right},
        )
    }

    // BFS out through every node
    for head := 0; head < len(queue); head++ {
        cur := queue[head]

        // -- de-dupe this node's recipes --
        cur.Node.Recipes = dedupeRecipes(cur.Node.Recipes)

        for ci, childRec := range cur.Node.Recipes {

            // de-dupe grandchildren as well before assigning IDs
            childRec.Left.Recipes  =
            ↪  dedupeRecipes(childRec.Left.Recipes)
            childRec.Right.Recipes =
            ↪  dedupeRecipes(childRec.Right.Recipes)

            childRec.Left.ID  = nextID()
            childRec.Right.ID = nextID()
            if updates != nil {
                updates <- Update{
                    Stage:       "startRecipe",
                    ElementName: cur.Node.Name,
                    Tier:        cur.Node.Tier,
                    ParentID:    cur.Node.ID,
                    RecipeIndex: ci,
                    LeftID:      childRec.Left.ID,
                    RightID:     childRec.Right.ID,
                    LeftLabel:   childRec.Left.Name,
                    RightLabel:  childRec.Right.Name,
                    Info:        fmt.Sprintf("recipe %d under
                    ↪  parent %d", ci, cur.Node.ID),
                }
            }
            queue = append(queue,
                item{ParentID: childRec.Left.ID, RecipeIndex: ci,
                ↪  Node: childRec.Left},
```

```
                item{ParentID: childRec.Right.ID, RecipeIndex: ci,
                ↪  Node: childRec.Right},
            )
        }
    }

    if updates != nil {
        updates <- Update{
            Stage:       "doneRecipe",
            ElementName: tgt.Name,
            Tier:        tgt.Tier,
            Info:        "completed BFS traversal",
        }
    }
    return tgt
}
```

Fungsi `BFS` digunakan untuk menemukan jalur-jalur komposisi menuju elemen target. Algoritma dimulai dengan mengelompokkan semua elemen ke dalam struktur `byTier` berdasarkan tingkat (*tier*) masing-masing. Tier 0 yang merepresentasikan elemen-elemen dasar langsung diinisialisasi ke dalam `memo` sebagai node tunggal, dan setiap elemen yang dimasukkan meningkatkan penghitung `nodeCounter`. Untuk tier yang lebih tinggi, pemrosesan dilakukan secara paralel menggunakan goroutine, channel semaphore, dan `sync.WaitGroup` untuk mengendalikan jumlah thread aktif. Setiap elemen akan diperiksa apakah dapat dikonstruksi dari dua elemen penyusun yang telah tersedia di `memo`. Jika valid, maka jalur komposisi baru akan ditambahkan ke `nextMemo`. Setelah semua elemen tier tersebut diproses, `nextMemo` digabungkan kembali ke dalam `memo`. Fungsi ini mengembalikan maksimal `maxPaths` buah jalur komposisi menuju `targetName`, disertai dengan jumlah total node yang telah dieksplorasi.

Fungsi `CreateFullTree` membentuk sebuah pohon resep akhir dari hasil-hasil jalur yang ditemukan oleh fungsi `BFS`. Fungsi ini menerima daftar jalur dalam bentuk `[]*Element`, serta channel `updates` opsional untuk pelaporan status progres secara real-time. Proses diawali dengan membuat objek `Target` dan menetapkan nama serta tier berdasarkan jalur pertama. Kemudian dilakukan proses kloning rekursif terhadap setiap jalur untuk menyusun struktur data baru yang bersih tanpa ID. Resep tingkat atas dari setiap jalur digabungkan berdasarkan kombinasi nama child kiri dan kanan menggunakan `map seen`. Selanjutnya, dilakukan traversal BFS untuk menetapkan ID unik bagi setiap node serta mengirimkan peristiwa pembaruan (`Update`) pada setiap resep yang dibentuk. Traversal ini dilakukan dengan menggunakan antrian `queue` secara *level-by-level* sesuai dengan definisi BFS dan tipe data bantu `item`. Setelah seluruh node diproses, fungsi mengembalikan objek `Target` komposit yang telah lengkap.

File: **dfs.go**

```go
/**
 *  A memoized recursive DFS returning how many full root → leaf
 ↪  chains an element can produce.
 *  It also updates the provided nodeCounter for tracking node
 ↪  exploration.
 */
func countPaths(recipeData map[string]scraper.ElementData, name
↪  string, nodeCounter *uint64) int {
    atomic.AddUint64(nodeCounter, 1)
    memo := make(map[string]int)
    var dfs func(string) int
    dfs = func(el string) int {
        atomic.AddUint64(nodeCounter, 1)
        if v, ok := memo[el]; ok {
            return v
        }
        data := recipeData[el]
        if data.Tier == 0 {
            memo[el] = 1
            return 1
        }
        sum := 0
        for _, rec := range data.Recipes {
            ln, rn := rec[0], rec[1]
            ld, lok := recipeData[ln]
            rd, rok := recipeData[rn]

            // skip if missing, tier-too-high, or uncraftable side
            if !lok || !rok ||
                ld.Tier >= data.Tier || rd.Tier >= data.Tier ||
                (lok && ld.Tier != 0 && len(ld.Recipes) == 0) ||
                (rok && rd.Tier != 0 && len(rd.Recipes) == 0) {
                continue
            }

            sum += dfs(ln) * dfs(rn)
        }
        memo[el] = sum
        return sum
    }
    return dfs(name)
}

/**
 *  BuildSubtreeInternal invokes DFSSearchInternal on one element
 ↪  up to maxPaths,
 *  Streaming its own nested events, and returns the built *Element
 ↪  + how many paths.
 */
func buildSubtreeInternal(
    recipeData map[string]scraper.ElementData,
    elementName string,
    maxPaths int,
    updateCh chan<- Update,
    nextID func() uint64,
```

```go
    forcedID uint64,
    nodeCounter *uint64,
) (*Element, int) {
    var subtree Tree
    DFS(recipeData, elementName, maxPaths, &subtree, updateCh,
    ↪  nextID, forcedID, nodeCounter)
    e := &Element{Name: subtree.Name, Tier: subtree.Tier, Recipes:
    ↪  subtree.Recipes, ID: subtree.ID}
    return e, subtree.UniquePaths
}

/**
 *  DFS is the main search function that builds a recipe tree for
 ↪  the target element.
 *  It returns the number of nodes explored during the search.
 */
func DFS(
    recipeData map[string]scraper.ElementData,
    targetName string,
    maxUniquePaths int,
    outPtr *Tree,
    updateCh chan<- Update,
    nextID func() uint64,
    forcedID uint64,
    nodeCounter *uint64,
) uint64 {
    data, ok := recipeData[targetName]
    if !ok {
        if updateCh != nil {
            updateCh <- Update{Stage: "error", ElementName:
            ↪  targetName, Tier: 0, Info: "not found"}
        }
        *outPtr = nil
        return *nodeCounter
    }
    id := forcedID
    if id == 0 {
        id = nextID()
    }
    root := &Target{Name: targetName, Tier: data.Tier, ID: id}
    *outPtr = root

    atomic.AddUint64(nodeCounter, 1)
    if updateCh != nil {
        updateCh <- Update{Stage: "startDFS", ElementName:
        ↪  targetName, Tier: data.Tier}
    }

    // -- Phase 1: parallel countPaths with pre-filter --
    numWorkers := runtime.NumCPU()
    type countJob struct {
        idx         int
        left, right string
    }
    type countRes struct {
        idx, leftCnt, rightCnt int
    }
```

```go
jobsCount := make(chan countJob, len(data.Recipes))
resultsCount := make(chan countRes, len(data.Recipes))
var wgCount sync.WaitGroup
wgCount.Add(numWorkers)
for w := 0; w < numWorkers; w++ {
    go func() {
        defer wgCount.Done()
        for job := range jobsCount {
            lc := countPaths(recipeData, job.left, nodeCounter)
            rc := countPaths(recipeData, job.right,
            ↪  nodeCounter)
            resultsCount <- countRes{job.idx, lc, rc}
        }
    }()
}

// enqueue only craftable-direct recipes
for i, rec := range data.Recipes {
    ln, rn := rec[0], rec[1]
    ld, lok := recipeData[ln]
    rd, rok := recipeData[rn]
    if !lok || !rok ||
        ld.Tier >= data.Tier || rd.Tier >= data.Tier ||
        (lok && ld.Tier != 0 && len(ld.Recipes) == 0) ||
        (rok && rd.Tier != 0 && len(rd.Recipes) == 0) {
        continue
    }
    jobsCount <- countJob{i, ln, rn}
}
close(jobsCount)
wgCount.Wait()
close(resultsCount)

type recInfo struct {
    left, right           string
    leftCount, rightCount int
}
infos := make([]recInfo, 0, len(data.Recipes))
for res := range resultsCount {
    rec := data.Recipes[res.idx]
    infos = append(infos, recInfo{rec[0], rec[1], res.leftCnt,
    ↪  res.rightCnt})
}

// -- Phase 2: parallel buildSubtreeInternal with shallow-leaf
↪  --
type buildJob struct {
    idx                   int
    leftName, rightName   string
    leftCount, rightCount int
    take                  int
}
type buildRes struct {
    idx                   int
    builtLeft, builtRight *Element
    usedPaths             int
```

```go
    }

    jobsBuild := make(chan buildJob, len(infos))
    resultsBuild := make(chan buildRes, len(infos))
    var wgBuild sync.WaitGroup
    wgBuild.Add(numWorkers)
    for w := 0; w < numWorkers; w++ {
        go func() {
            defer wgBuild.Done()
            for job := range jobsBuild {
                parentID := root.ID
                leftID := nextID()
                rightID := nextID()

                if updateCh != nil {
                    updateCh <- Update{
                        Stage:       "startRecipe",
                        ElementName: targetName,
                        Tier:        data.Tier,
                        RecipeIndex: job.idx,
                        Info:        fmt.Sprintf("%s + %s → take
                        ↪  %d", job.leftName, job.rightName,
                        ↪  job.take),
                        ParentID:    parentID,
                        LeftID:      leftID,
                        RightID:     rightID,
                        LeftLabel:   job.leftName,
                        RightLabel:  job.rightName,
                    }
                }

                var leftElem, rightElem *Element
                var used int

                if job.leftCount == 0 || job.rightCount == 0 {
                    ld := recipeData[job.leftName]
                    rd := recipeData[job.rightName]
                    leftElem = &Element{Name: job.leftName, Tier:
                    ↪  ld.Tier, ID: leftID}
                    rightElem = &Element{Name: job.rightName, Tier:
                    ↪  rd.Tier, ID: rightID}
                    used = 0

                } else {
                    full := job.leftCount * job.rightCount
                    var leftNeed, rightNeed int
                    if job.take == full {
                        leftNeed, rightNeed = job.leftCount,
                        ↪  job.rightCount
                    } else {
                        leftNeed = (job.take + job.rightCount - 1)
                        ↪  / job.rightCount
                        if leftNeed > job.leftCount {
                            leftNeed = job.leftCount
                        }
                        rightNeed = job.take
                    }
```

```
                    leftElem, _ = buildSubtreeInternal(recipeData,
                    ↪  job.leftName, leftNeed, updateCh, nextID,
                    ↪  leftID, nodeCounter)
                    rightElem, _ = buildSubtreeInternal(recipeData,
                    ↪  job.rightName, rightNeed, updateCh, nextID,
                    ↪  rightID, nodeCounter)

                    used = job.take
                }

                resultsBuild <- buildRes{job.idx, leftElem,
                ↪  rightElem, used}
            }
        }()
    }

    // Schedule build jobs (always include the recipe, but prune to
    ↪  remaining)
    remaining := maxUniquePaths
    for idx, info := range infos {
        if remaining <= 0 {
            break
        }
        total := info.leftCount * info.rightCount
        take := total
        if take > remaining {
            take = remaining
        }
        remaining -= take
        jobsBuild <- buildJob{idx, info.left, info.right,
        ↪  info.leftCount, info.rightCount, take}
    }
    close(jobsBuild)
    wgBuild.Wait()
    close(resultsBuild)

    // Collect & attach
    for br := range resultsBuild {
        root.Recipes = append(root.Recipes, Recipe{Left:
        ↪  br.builtLeft, Right: br.builtRight})
        root.UniquePaths += br.usedPaths
    }

    return *nodeCounter
}
```

Fungsi `countPaths` adalah fungsi rekursif yang digunakan untuk menghitung jumlah jalur penuh dari elemen akar ke daun dalam pohon resep. Fungsi ini bekerja dengan cara melakukan pencarian rekursif untuk setiap pasangan resep yang ditemukan dalam data resep. Fungsi ini juga memanfaatkan teknik memoization untuk menyimpan hasil perhitungan sebelumnya guna menghindari perhitungan ulang yang tidak perlu. Teknik ini meningkatkan efisiensi eksekusi, terutama untuk elemen yang sering dipanggil berulang kali.

Pada implementasinya, setiap kali sebuah elemen diproses, penghitung `nodeCounter` diperbarui menggunakan `atomic.AddUint64`, untuk memastikan sinkronisasi update dari `nodeCounter`.

Fungsi `buildSubtreeInternal` adalah fungsi yang membangun subpohon untuk sebuah elemen tertentu, dengan cara memanggil fungsi `DFS` untuk membangun subpohon dari elemen target. Fungsi ini juga mengirimkan update melalui channel `updateCh` yang berisi status dan informasi terkait perhitungan dan pembangunan subpohon. Pembangunan subpohon dilakukan secara paralel menggunakan teknik multithreading, dimana pekerjaan dipartisi menjadi tugas-tugas yang lebih kecil yang kemudian dieksekusi oleh beberapa thread secara bersamaan.

Pada fase pertama dari eksekusi, fungsi `DFS` melakukan perhitungan jumlah jalur untuk setiap pasangan resep dalam elemen yang sedang diproses. Teknik multithreading diterapkan dengan cara memanfaatkan `sync.WaitGroup` untuk mengelola sinkronisasi eksekusi antara beberapa thread yang melakukan perhitungan jalur secara paralel. `runtime.NumCPU()` digunakan untuk menentukan jumlah thread yang akan dijalankan, yang bergantung pada jumlah inti CPU yang tersedia pada sistem. Channel `jobsCount` digunakan untuk antrian pekerjaan perhitungan, sementara channel `resultsCount` digunakan untuk mengumpulkan hasil dari setiap perhitungan jalur.

Pada fase kedua, fungsi `DFS` melanjutkan dengan membangun subpohon untuk setiap resep yang diproses. Untuk setiap pasangan resep, tugas pembangunan subpohon dijadwalkan dan dieksekusi oleh beberapa thread secara paralel. Channel `jobsBuild` digunakan untuk menjadwalkan pekerjaan, sementara channel `resultsBuild` mengumpulkan hasil pembangunan subpohon. Penggunaan `sync.WaitGroup` kembali digunakan untuk menunggu semua thread selesai sebelum melanjutkan.

4.2. Alur Program

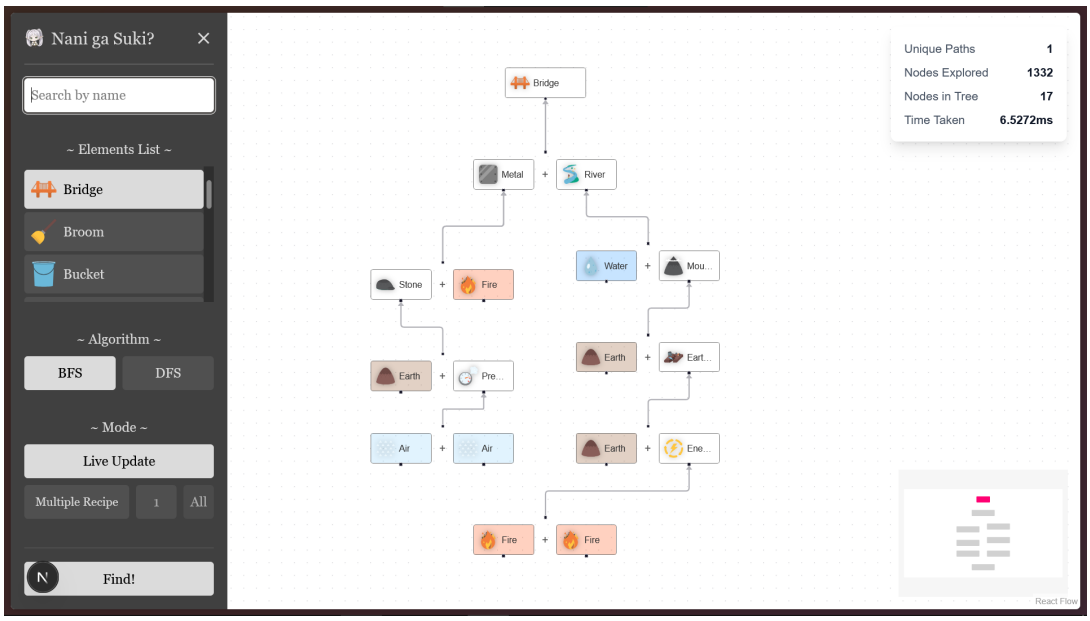
Berikut adalah tata cara penggunaan website Nani ga Suki?.

1. Pengguna mengetikkan nama atau sebagian nama elemen pada kotak teks pencarian yang terletak di bagian atas Sidebar.
2. Daftar *element* yang tersedia akan terfilter secara *real-time* berdasarkan masukan pengguna filter nama elemen dari pengguna.
3. Di bawah kotak teks pencarian, pengguna dapat memilih salah satu elemen yang ingin dicari resepnya.
4. Pengguna dapat memilih mode *single/multiple recipe* dan memasukkan jumlah *recipe* yang diinginkan. Pengguna juga dapat mengatur *toggle live update*.
5. Setelah menekan tombol ”Find!” akan terbentuk *path recipe* yang diinginkan. Jika pengguna memerlukan ruang tampilan yang lebih luas untuk melihat diagram hierarki, dapat menyembunyikan Sidebar dengan menekan *hamburger menu* yang terletak di sudut kiri atas layar.

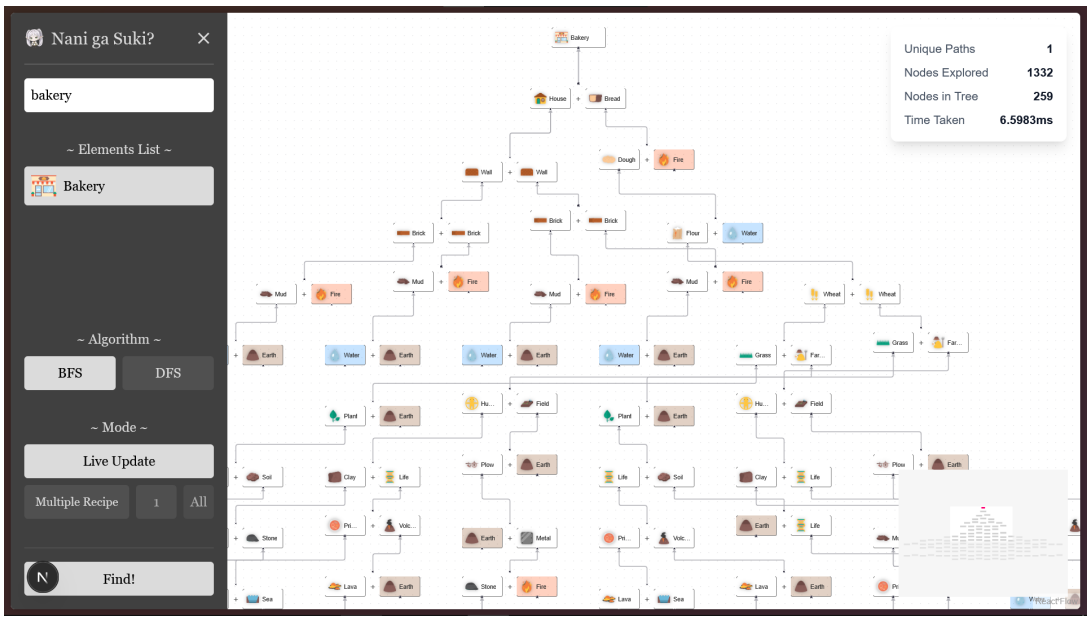
4.3. Eksperimen

Berikut hasil eksperimen dari berbagai tes yang diujikan, mulai *test-case* umum hingga *stress-case* untuk menilai performa program.

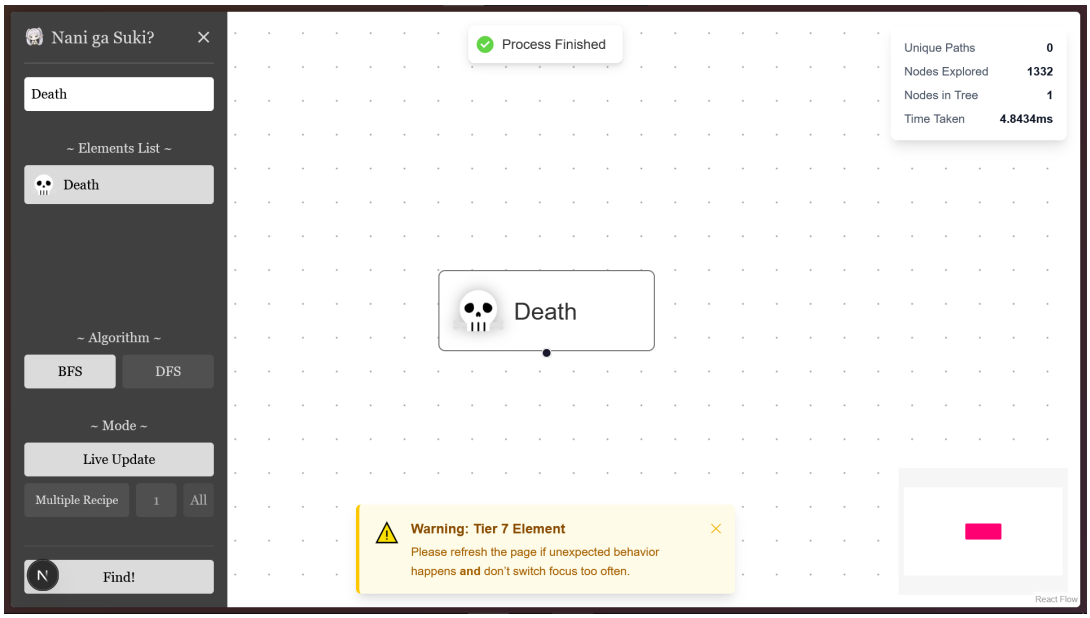
a. Single Recipe BFS



Gambar 12: Percobaan Single Recipe BFS pada Elemen Tier 5

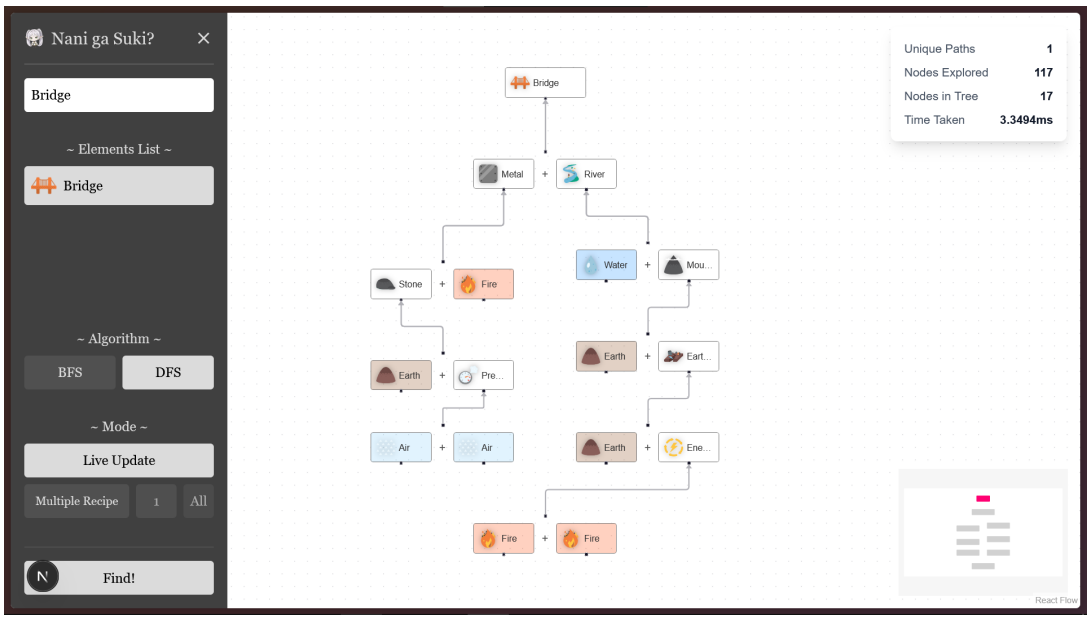


Gambar 13: Percobaan Single Recipe BFS pada Elemen Tier 14

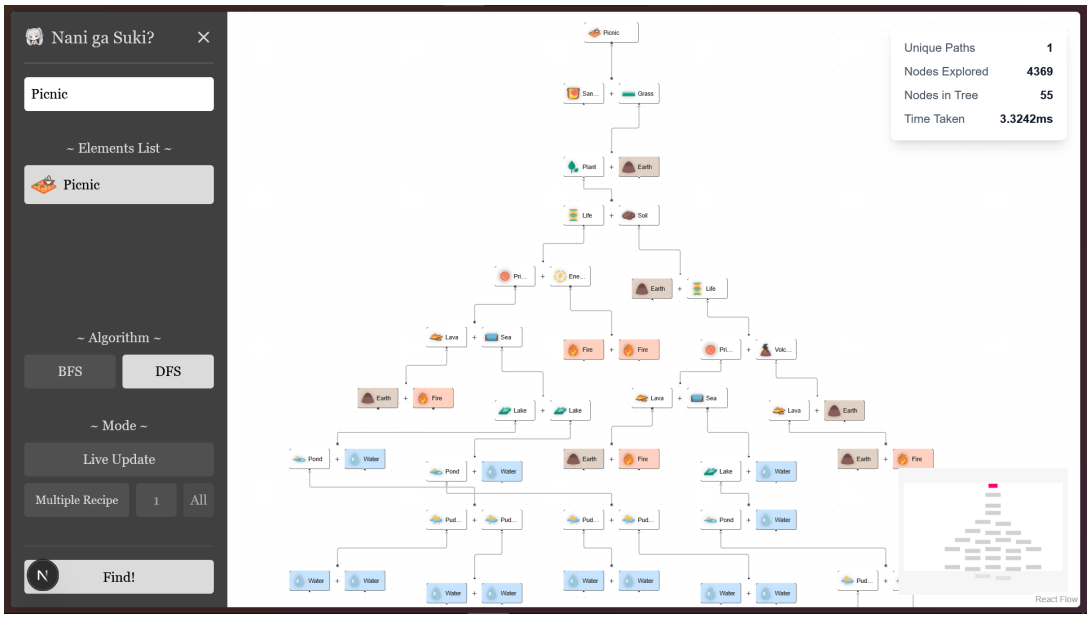


Gambar 14: Percobaan Single Recipe BFS pada Elemen *Uncraftable*

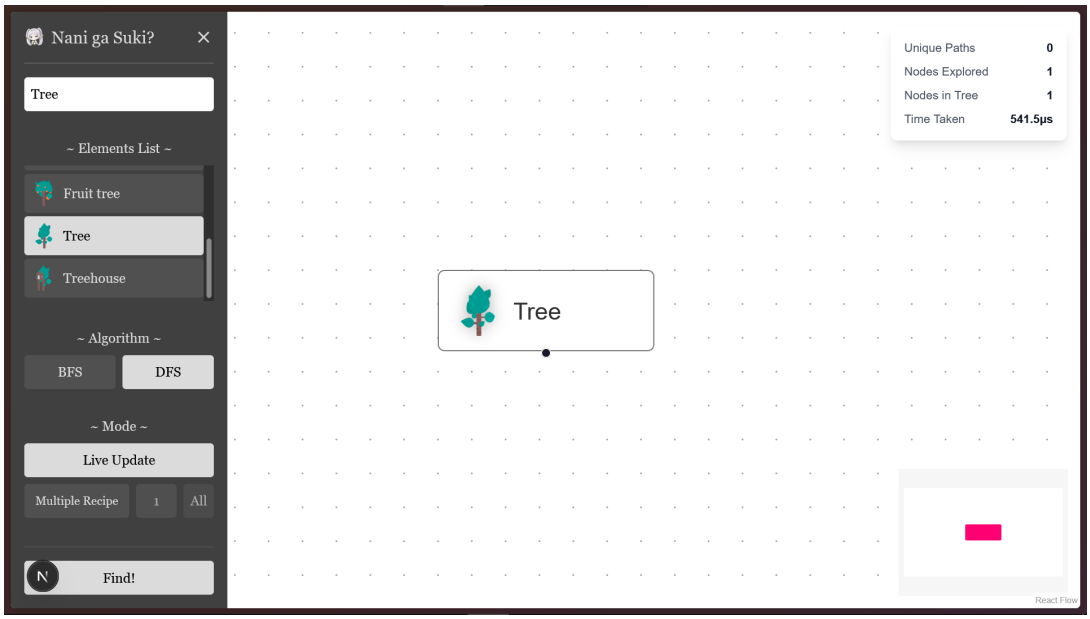
b. Single Recipe DFS



Gambar 15: Percobaan Single Recipe DFS pada Elemen Tier 3

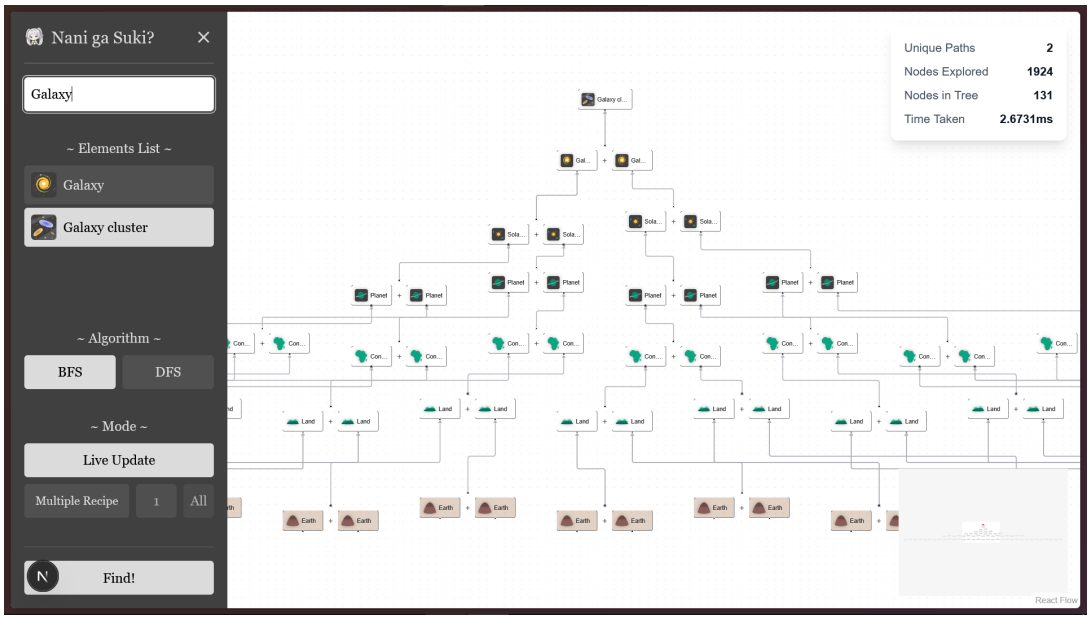


Gambar 16: Percobaan Single Recipe DFS pada Elemen Tier 15

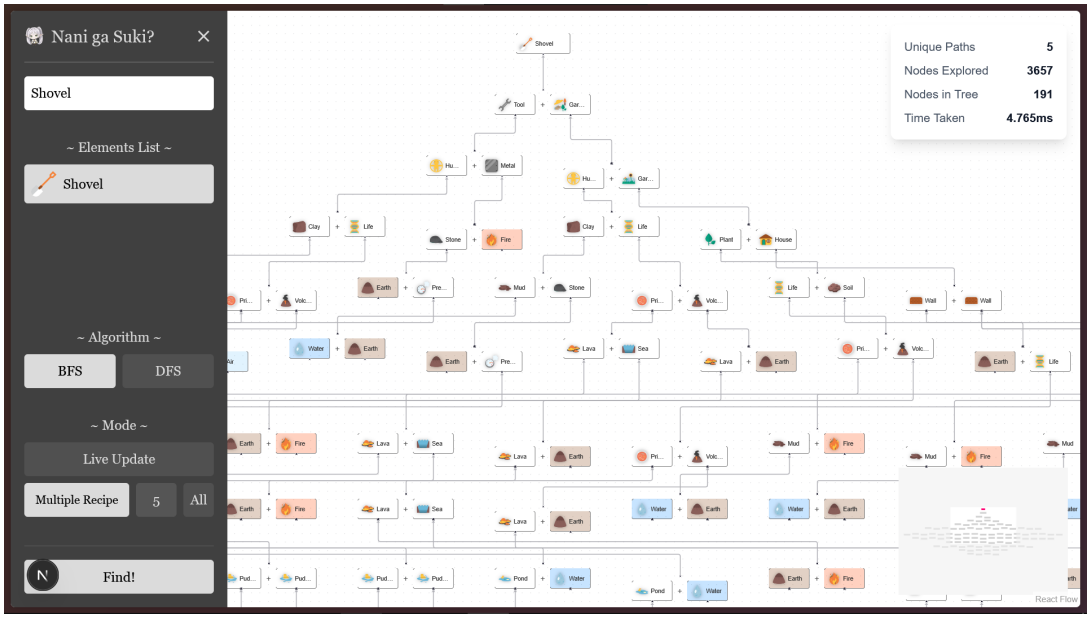


Gambar 17: Percobaan Single Recipe DFS pada Elemen *Uncraftable*

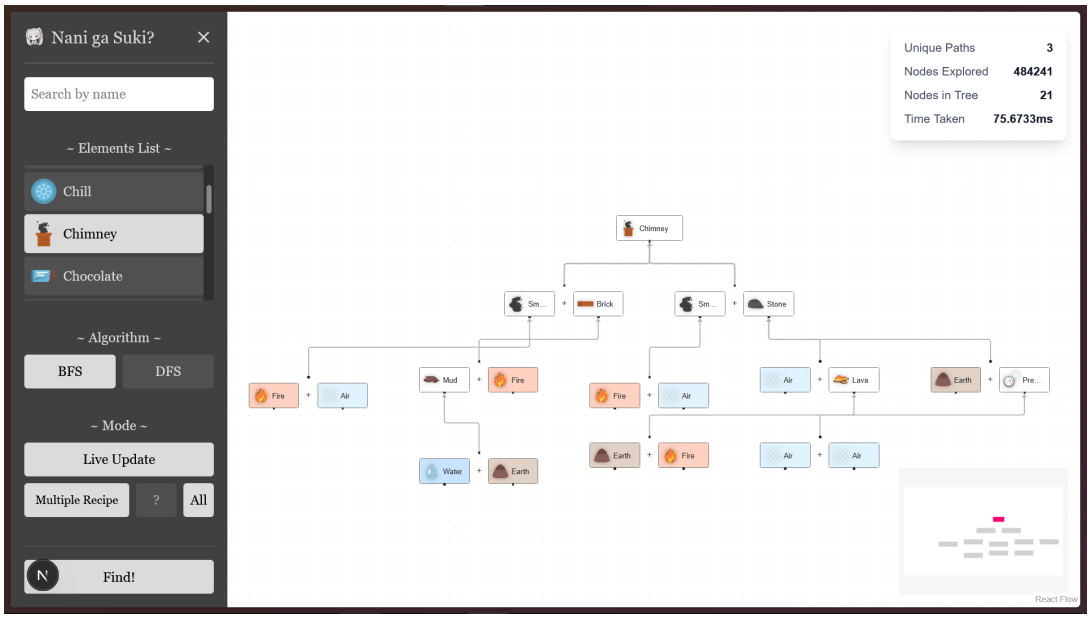
c. Multiple Recipe BFS



Gambar 18: Percobaan Multiple Recipe BFS pada Elemen Tier 6

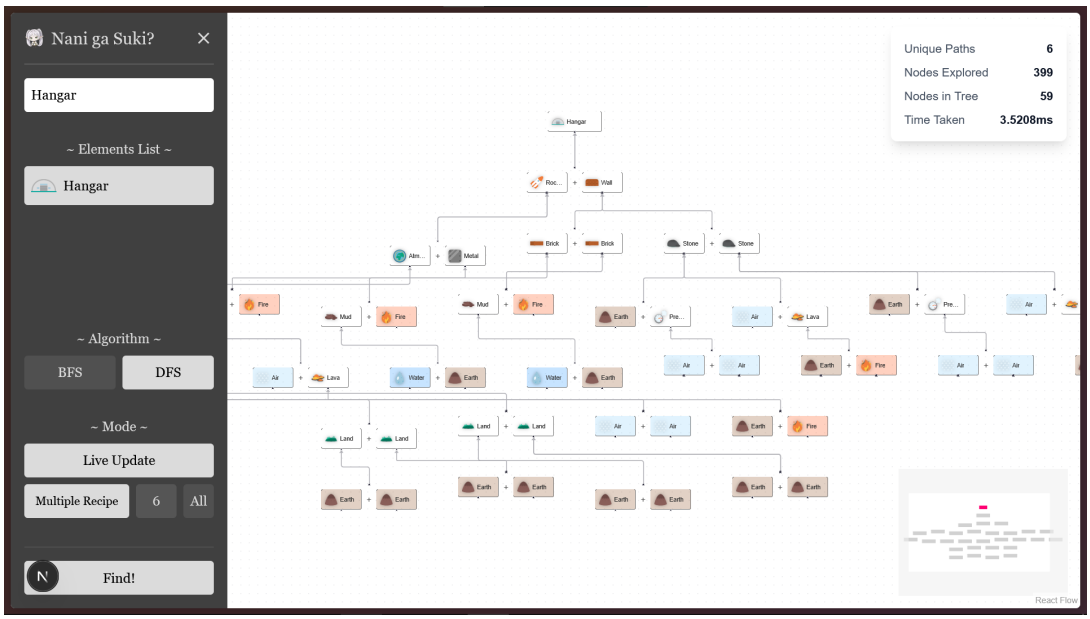


Gambar 19: Percobaan Multiple Recipe BFS pada Elemen Tier 11

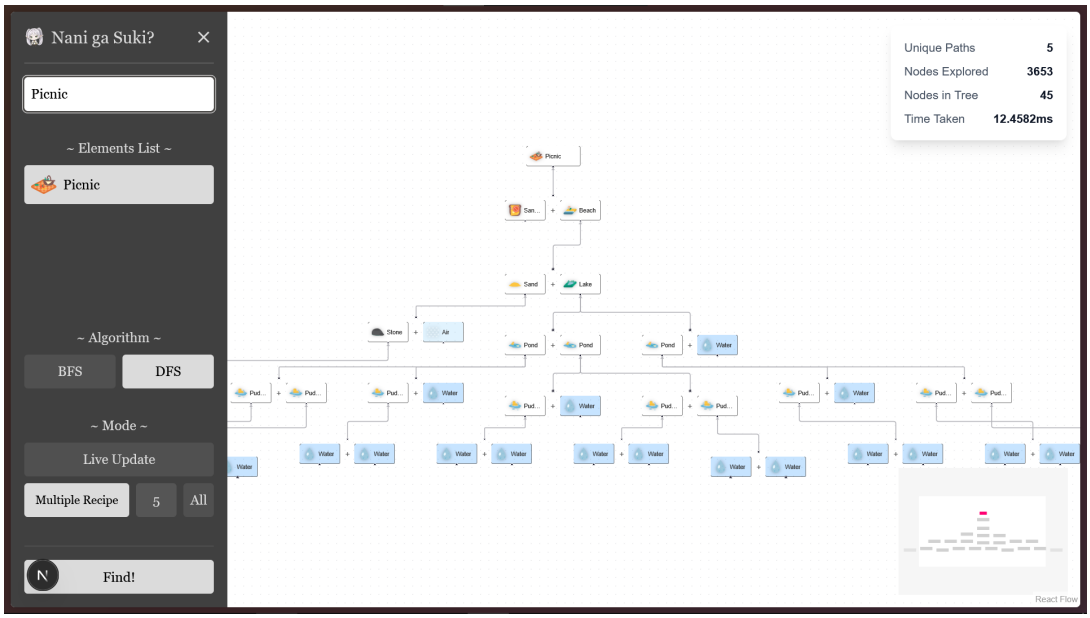


Gambar 20: Percobaan Multiple Recipe BFS dengan tombol "All"

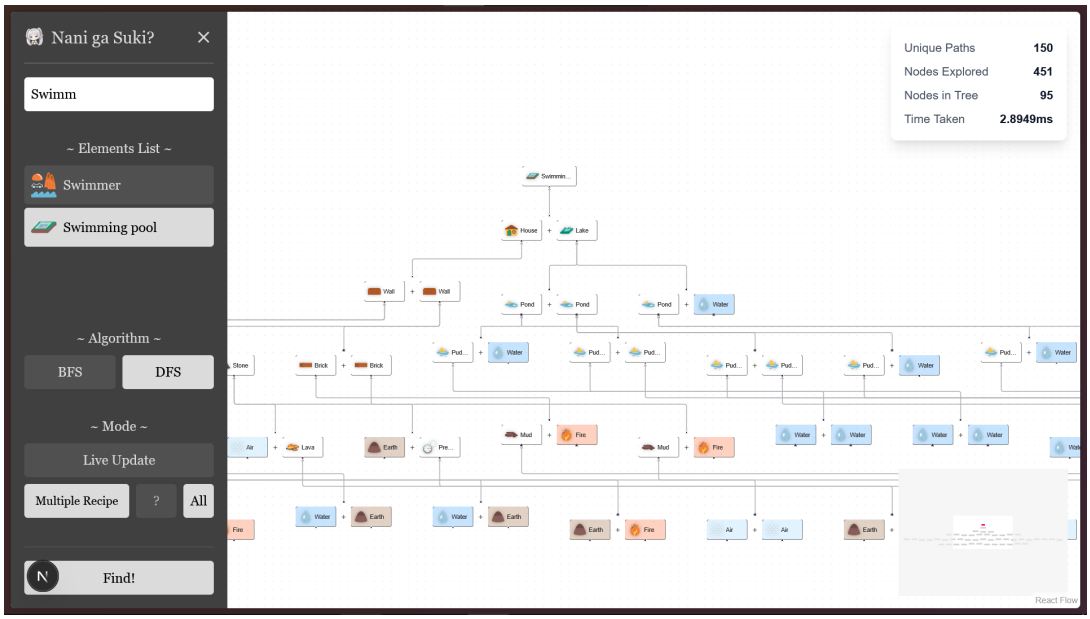
d. Multiple Recipe DFS



Gambar 21: Percobaan Multiple Recipe DFS pada Elemen Tier 3



Gambar 22: Percobaan Multiple Recipe DFS pada Elemen Tier 15



Gambar 23: Percobaan Multiple Recipe DFS pada tombol "All"

4.4. Analisis dan Pembahasan

Secara teori, algoritma BFS cenderung menggunakan lebih banyak memori dibandingkan DFS. Hal ini disebabkan oleh sifat BFS yang menyimpan semua node pada level yang sama dalam antrian untuk diproses secara berurutan. Oleh karena itu, jumlah node yang dikunjungi pada BFS jauh lebih banyak daripada DFS, yang hanya mengeksplorasi satu cabang dalam pohon secara mendalam.

Meskipun BFS secara teori diharapkan menghasilkan jalur yang lebih pendek (karena pencarian dilakukan secara level per level), eksperimen yang dilakukan menunjukkan bahwa panjang jalur yang ditemukan oleh BFS dan DFS pada praktiknya cukup serupa. Hal ini mengindikasikan bahwa meskipun BFS lebih eksploratif dalam mencari jalur terpendek, karakteristik dari permasalahan (dalam kasus ini, game Little Alchemy) mempengaruhi hasil akhirnya, sehingga keduanya menghasilkan jalur dengan panjang yang serupa.

Secara umum, BFS memiliki keunggulan dalam menemukan jalur terpendek di dalam graf, karena algoritma ini memproses node secara merata berdasarkan tingkat kedalamannya. Namun, BFS membutuhkan lebih banyak memori karena harus menyimpan semua node pada level tertentu dalam antrian untuk diproses lebih lanjut.

Di sisi lain, DFS lebih efisien dalam hal penggunaan memori. Karena traversal dilakukan secara mendalam pada cabang-cabang tertentu, DFS tidak perlu menyimpan node sebanyak yang dilakukan oleh BFS, yang memungkinkan algoritma ini lebih hemat memori. Namun, DFS tidak selalu memberikan jalur terpendek, karena eksplorasi dilakukan lebih dalam pada cabang-cabang tertentu dan tidak memperhitungkan jarak antar node secara keseluruhan.

Secara keseluruhan, baik BFS maupun DFS bekerja dengan baik untuk menyelesaikan masalah pencarian jalur pada pohon atau graf, dengan kelebihan dan kekurangannya masing-masing. Pilihan antara BFS dan DFS sangat bergantung pada karakteristik graf yang dihadapi, ukuran pohon, dan kebutuhan untuk menemukan jalur yang optimal. Keduanya dapat digunakan dengan efektif sesuai dengan konteks permasalahan yang ada.

Bab V
Penutup

5.1. Kesimpulan

Dalam proyek Tugas Besar 2 IF2211 Strategi Algoritma ini, algoritma BFS dan DFS dapat digunakan dalam proses pencarian *recipe* pada permainan Little Alchemy 2. Kami berhasil untuk mengimplementasikan algoritma BFS dan DFS dengan struktur data persoalan menjadi *Tree* dalam *website* pencarian *recipe* kami. Selain menghasilkan *full path* dari *element* dasar menuju *element* yang diinginkan, program kami juga dapat menghasilkan `Live Update` yang dapat memberi *insight* yang lebih mendalam terhadap bagaimana masing-masing algoritma BFS dan DFS bekerja dalam program pencarian *recipe* kami.

Selain itu, pengguna dari Internet memulai *website* yang telah di-*deploy* dengan cepat dan mudah untuk mendapatkan *recipe-recipe* dari *element* yang diinginkan ketika sedang kesusahan untuk mencari *recipe* saat bermain Little Alchemy 2 . Kemudian, terdapat tombol `all` juga yang akan menampilkan seluruh *recipe path* yang mungkin dari suatu *element*, membuat fleksibilitas program dalam menerima perintah dari pengguna.

5.2. Saran

Pelaksanaan Tugas Kecil 2 IF2211 Strategi Algoritma di Semester II (Genap) Tahun 2024/2025 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

rzi

Minimal revisi spek utama jangan H-4 banh :(

kewz

ez

notlqmn

Tubes ini sangat menantang, baik secara akademis maupun secara mental. Godaan *long weekend* menjadi tantangan yang cukup berat. Terbukti dari ke-*mager*-an saya dan ”rekan-rekan” saya saat mengerjakan maupun memonitor tubes ini. Saya mengingatkan diri saya dan semua rekan-rekan agar tetap menjaga motivasi dan semangat belajar di tengah *long weekend* atau libur panjang lainnya. Walaupun hari libur adalah waktu untuk istirahat, hari libur tidak serta-merta membuat tanggung jawab dan kewajiban kita menghilang sementara.

Semoga saran-saran ini membantu pembaca dalam menyiapkan diri untuk menangani tugas serupa di masa depan.

5.3. Refleksi

Ruang perbaikan dan pengembangan dalam mengerjakan tugas dapat difokuskan pada beberapa aspek yang dapat ditingkatkan. Pertama-tama, pengaturan waktu menjadi kunci utama. Kami menyadari bahwa perencanaan waktu yang lebih baik dapat meningkatkan efisiensi pengerjaan. Menerapkan strategi manajemen waktu, seperti membuat jadwal yang terstruktur dan menetapkan tenggat waktu internal untuk setiap tahap pekerjaan, dapat membantu menghindari tekanan waktu yang tidak perlu.

Selain itu, ketelitian membaca spesifikasi dari awal menjadi aspek yang perlu diperhatikan. Memahami secara menyeluruh tentang apa yang diminta dalam tugas, termasuk detail-detail kecil, dapat mengurangi risiko kesalahan dan memastikan pekerjaan berjalan sesuai dengan harapan. Menempatkan perhatian ekstra pada spesifikasi tugas dapat meminimalkan revisi dan penyesuaian yang mungkin diperlukan.

Komunikasi tim yang baik juga dapat dioptimalkan. Membuat saluran komunikasi yang jelas dan terbuka di antara anggota tim dapat menghindari kebingungan dan memastikan bahwa semua anggota memiliki pemahaman yang sama tentang tujuan dan tanggung jawab masing-masing. Diskusi reguler dan pembahasan mengenai kemajuan proyek dapat meningkatkan keterlibatan semua anggota tim.

Dalam konteks pengembangan web, penambahan fitur-fitur yang mendukung dapat memberikan nilai tambah. Memperhatikan kebutuhan pengguna dan mengidentifikasi area yang dapat diperbaiki atau ditingkatkan dalam hal fungsionalitas dapat meningkatkan kualitas tugas. Pemikiran kreatif dalam mengimplementasikan fitur-fitur baru yang relevan dengan tujuan proyek dapat membuat proyek lebih bermanfaat dan menarik.

Dalam konteks optimalisasi algoritma, dapat melakukan eksplorasi lebih jauh lagi mengenai efisiensi kinerja program. Dengan memahami cara agar kinerja program lebih efisien, program akan menjadi lebih cepat dan lebih baik lagi. Hal tersebut sangat diperlukan agar program kami bisa menjadi program yang layak untuk dipublikasikan.

Terakhir, evaluasi diri secara berkala dapat menjadi langkah penting. Menerima umpan balik, baik dari anggota tim maupun asisten, dan memanfaatkannya sebagai dasar untuk perbaikan lebih lanjut adalah cara efektif untuk terus berkembang. Selalu terbuka terhadap saran dan berkomitmen untuk belajar dari setiap pengalaman dapat membantu memperbaiki kinerja secara berkelanjutan.

Lampiran

Repository, *Website*, dan Video program pada Tugas Besar 2 ini dapat diakses melalui tautan berikut:

- **GitHub:**

  `https://github.com/zirachw/Tubes2_SeleniumSoup4`

- **Website:**

  `http://nanigasuki.xyz`

- **Video:**

  `https://linktr.ee/Zirach`

Adapun berikut tabel spesifikasi dari Tugas Besar 2 ini:

Tabel 1: Tabel Spesifikasi Tuges Besar 2

No	Poin	Ya	Tidak
1	Aplikasi dapat dijalankan.	✓	
2	Aplikasi dapat memperoleh data *recipe* melalui scraping.	✓	
3	Algoritma *Depth First Search* dan *Breadth First Search* dapat menemukan *recipe* elemen dengan benar.	✓	
4	Aplikasi dapat menampilkan visualisasi *recipe* elemen yang dicari sesuai dengan spesifikasi.	✓	
5	Aplikasi mengimplementasikan multithreading.	✓	
6	Membuat laporan sesuai dengan spesifikasi.	✓	
7	Membuat bonus video dan diunggah pada Youtube.	✓	
8	Membuat bonus algoritma pencarian *Bidirectional*.		✓
9	Membuat bonus *Live Update*.	✓	
10	Aplikasi di-*containerize* dengan Docker.	✓	
11	Aplikasi di-*deploy* dan dapat diakses melalui internet.	✓	

Referensi

Rinaldi Munir. (2025). "Breadth First Search (BFS) dan Depth First Search (DFS) - Bagian 1 (Versi baru 2025)." Diakses pada 10 Mei 2025. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-(2025)-Bagian1.pdf

Rinaldi Munir. (2025). "Breadth First Search (BFS) dan Depth First Search (DFS) - Bagian 2 (Versi baru 2025)." Diakses pada 10 Mei 2025. https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-(2025)-Bagian2.pdf

Geeks For Geeks. (2025). "Breadth First Search or BFS for a Graph." Diakses pada 11 Mei 2025. https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/

M. Fathur Rizky (2025). "Depth-First Search dan Breadth-First Search" Diakses pada 11 Mei 2025. https://medium.com/@fathurwithyou/depth-first-search-dan-breadth-first-search-66ca5cae0289

Berman E. (2024). "Depth-First Search (DFS) Algorithm" Diakses pada 11 Mei 2025. https://medium.com/@that-software-PM/depth-first-search-dfs-algorithm-201dc95e524

PuerkitoBio. (2025). "`goquery` Library" Diakses pada 8 Mei 2025. https://github.com/PuerkitoBio/goquery

xyflow. (2025). "`reactflow` Library" Diakses pada 8 Mei 2025. https://github.com/xyflow/xyflow